



Qualcomm Linux Camera Guide

80-80021-17 AD

March 24, 2026

About Qualcomm

Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Building on our 40 years of technology leadership in creating era-defining breakthroughs, we deliver a broad portfolio of solutions built with our leading-edge AI, high-performance, low-power computing, and unrivaled connectivity. Our Snapdragon® platforms power extraordinary consumer experiences, and our Qualcomm Dragonwing™ products empower businesses and industries to scale to new heights. Together with our ecosystem partners, we enable next-generation digital transformation to enrich lives, improve businesses, and advance societies. At Qualcomm, we are engineering human progress.

Contents

1	Camera documentation	3
1.1	Camera overview	3
1.2	Stream cameras	3
1.3	Enhance camera output	3
1.4	Troubleshoot camera	4
2	Camera overview	5
2.1	Camera components	5
2.2	Prerequisites	7
3	Stream cameras	8
3.1	Set up the camera	8
3.1.1	Connect RB3G2 MIPI CSI on vision mezzanine board	9
3.1.2	Set SW2300 DIP switch on interposer board	10
3.1.3	Connect RB3G2 GMSL cameras on vision mezzanine board	10
3.1.4	Connect RB3G2 MIPI CSI cameras on interposer board	11
3.1.5	Supported resolutions and features	12
3.1.6	Connect MIPI CSI cameras on RB8	14
3.1.7	Connect GMSL cameras on RB8	15
3.1.8	Set DIP switches	16
3.1.9	Supported resolutions and features	17
3.1.10	Concurrent camera support on RB8	17
3.1.11	Connect MIPI CSI camera module on RB4	20
3.1.12	Connect GMSL camera module on RB4	21
3.1.13	Set DIP switches	22
3.1.14	Supported resolutions and features	23
3.1.15	Concurrent camera support on RB4	23
3.1.16	IQ615 EVK board switch configuration	29
3.2	Choose the stream API	29
3.3	Stream camera with the GStreamer API	29
3.3.1	Prerequisites	30
3.3.2	Run camera use cases	30
	Single camera stream start	30
	Video encoding	31
	Video encoding and snapshot	31

	MIPI multi-camera streaming	33
	Single camera stream start	35
	Video encoding	35
	Video encoding and snapshot	36
	MIPI multi-camera streaming	37
	GMSL multi-camera streaming	37
	Single camera stream start	38
	Video encoding	38
	Video encoding and snapshot	39
	MIPI multi-camera streaming	40
	GMSL multi-camera streaming	40
3.3.3	Other GStreamer Samples	41
3.3.4	Stream camera with the V4L2 API	41
	CAMSS driver	42
	V4L2 sample application - Yavta	44
	V4L2 sample application - libcamera	47
	Capture raw frame data	47
	CAMSS driver	49
	V4L2 sample application - Yavta	51
	V4L2 sample application - libcamera	54
	Capture raw frame data	54
	CAMSS driver	56
	V4L2 sample application - Yavta	58
	V4L2 sample application - libcamera	61
	Capture raw frame data	61
	CAMSS driver	63
	V4L2 sample application - Yavta	65
	V4L2 sample application - libcamera	68
	Capture raw frame data	68
3.4	USB camera	70
3.5	Network camera	70
4	Enhance camera output	71
4.1	Support multi-camera using offline IFE	71
	4.1.1 Test procedure	72
	4.1.2 Log verification	73
4.2	Support per port grouping	73
	4.2.1 Validation	75
4.3	Support software TNR/MCTF	79
4.4	Enable high dynamic range	80
	4.4.1 SHDR v3 - Dual virtual channel with two-frame exposure	81
	4.4.2 SHDR v2 - Single virtual channel with two-frame exposure	82
	4.4.3 Defog feature	82
4.5	Enable EIS and LDC	84
	4.5.1 EIS single stream use case	85

4.5.2	EIS enabled on first of two streams use case	85
4.5.3	EIS enabled on two streams	86
4.5.4	LDC single stream use case	87
4.5.5	LDC enabled on two streams use case	87
4.5.6	EIS and LDC single stream use case	88
4.5.7	EIS and LDC enabled on first of two streams use case	89
4.5.8	EIS and LDC enabled on two streams use case	89
4.6	Advanced feature concurrences	90
4.6.1	SHDR v3 and EIS concurrency use case	91
4.6.2	SHDR v3 and LDC concurrency use case	92
4.6.3	SHDR v2 and LDC concurrency use case	93
4.6.4	SHDR v3, EIS, and LDC concurrency use case	93
4.7	Enable multiple ROI streams	94
4.8	Switch linear vs. SHDR mode automatically	97
5	Troubleshoot camera	99
5.1	Capture user mode logs	99
5.1.1	Camera UMD logs	100
5.1.2	Camera user mode logging groups	101
5.2	Capture kernel mode logs	101
5.2.1	Enable camera kernel mode driver logs	102
5.2.2	Enable CSIPHY log dumps	104
5.2.3	Enable CSID SOF/EOF IRQ logs	104
5.2.4	Collect CPAS logs	105
5.2.5	Enable kernel run-time log mask	105
6	References	107
6.1	Related documents	107

1 Camera documentation

Connect camera sensors, enable camera use cases, and learn about available advanced features.

1.1 Camera overview

[Camera components](#)

Review the components of the camera subsystem.

[Prerequisites](#)

Set up your device for baseline camera functionality.

1.2 Stream cameras

[Set up the camera](#)

Connect camera sensors to your reference hardware platform.

[Choose a stream API](#)

Review the available stream API options.

[Stream with the GStreamer API](#)

Use GStreamer APIs to stream cameras.

[Stream with the V4L2 API](#)

Use the the V4L2 framework to stream cameras.

1.3 Enhance camera output

[Enable high dynamic range](#)

Enable the staggered high dynamic range (SHDR) feature.

[Enable EIS and LDC](#)

Enable the electronic image stabilization (EIS) and lens distortion correction (LDC) features.

[Enable defog](#)

Enable the defog feature.

1.4 Troubleshoot camera

[Troubleshoot sensor streaming](#)

Troubleshoot camera issues.

2 Camera overview

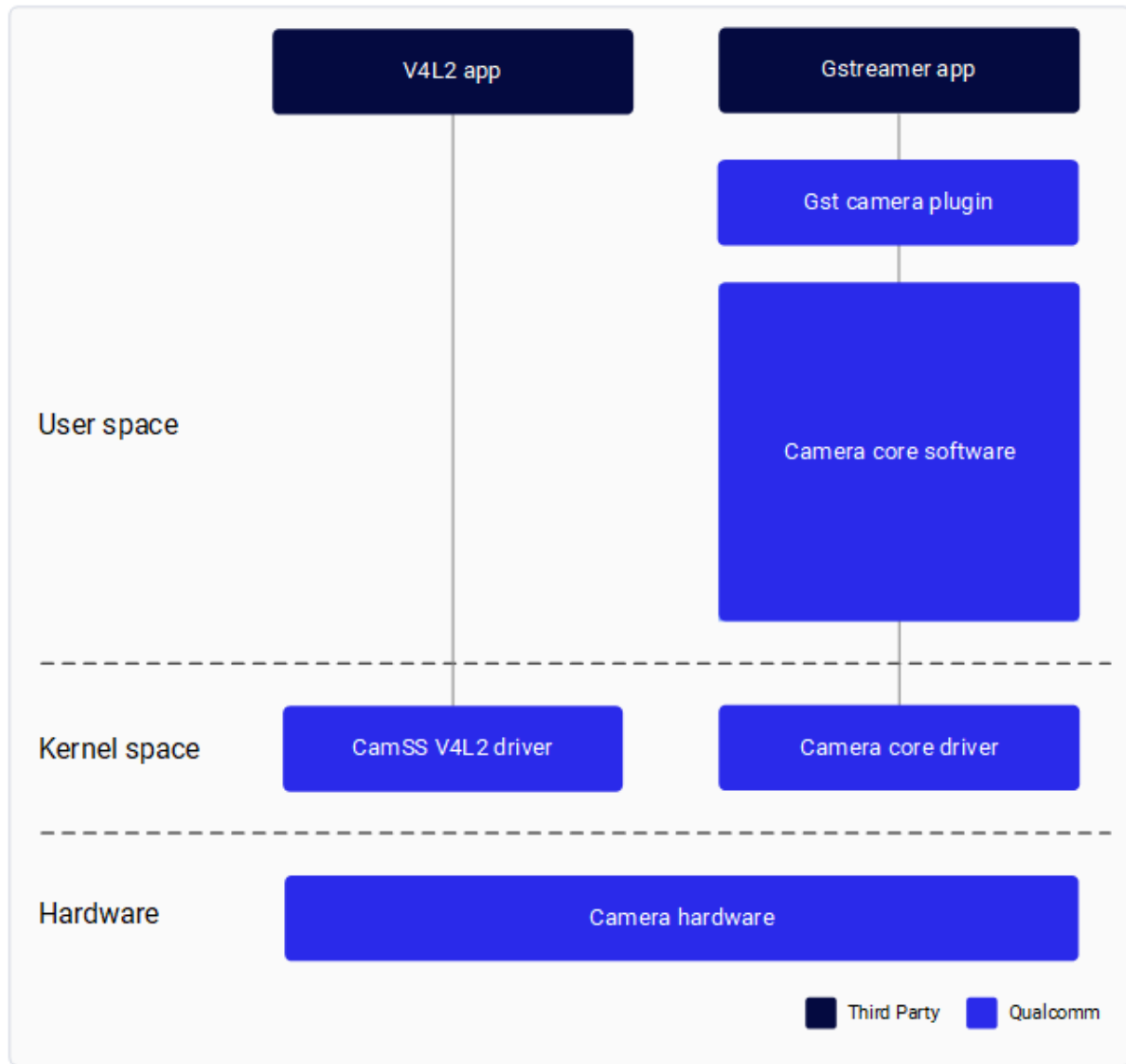
This document explains the camera subsystem that receives data through a MIPI CSI interface.

USB camera data is delivered through a USB interface, and the camera subsystem doesn't involve USB camera data transaction. See [USB camera configuration](#) for USB camera usage.

Network camera data is delivered through a network interface, and the camera subsystem doesn't involve network camera data transaction. The device can receive network camera data through the GStreamer [rtspsrc](#) plugin.

2.1 Camera components

The following diagram shows the components of Qualcomm Camera.



The following components are provided by Qualcomm:

Component	Description
Gst camera plugin (qtiqmmfsrc)	GStreamer plugin for Qualcomm's camera subsystem
Camera Core Software	Qualcomm's proprietary camera software that provides the interface to develop camera sensor drivers, camera tuning, and custom software nodes
Camera Core Driver	Qualcomm camera subsystem driver in the downstream Linux kernel
CamSS V4L2 driver	Qualcomm camera subsystem driver in the upstream Linux kernel

Qualcomm provides a GStreamer plug-in to enable application developers to interface with the Qualcomm camera subsystem. See [GStreamer API](#) for application development.

Qualcomm provides an interface for camera sensor driver developers and IQ tuning engineers to develop their own sensor drivers and perform custom IQ tuning. See Camera Sensor Driver Development and Tuning in the [Camera Addendum document](#).

Access to the meta- qcom-extras layer is required for camera sensor driver development and custom IQ tuning. For the access level, refer to [Mapping access levels](#).

2.2 Prerequisites

- Set up your infrastructure as described in the [Qualcomm Linux Build Guide](#)
- Flash the latest software release to the development board
- Set up SSH connection:
 1. Enable SSH in Permissive mode by performing the steps mentioned in [Use SSH](#).
 2. Connect to the device by running the following command:

```
ssh root@<device_IP_address>
```

For example, if the IP address of the device is 10.92.160.222, run the following command:

```
ssh root@10.92.160.222
```

- The downstream camera driver isn't enabled by default. To activate it, refer to the camera overlay configuration steps described in [Qualcomm Linux Migration Guide](#).

3 Stream cameras

This page describes how to connect camera sensors to your reference hardware platform and provides information about available APIs.

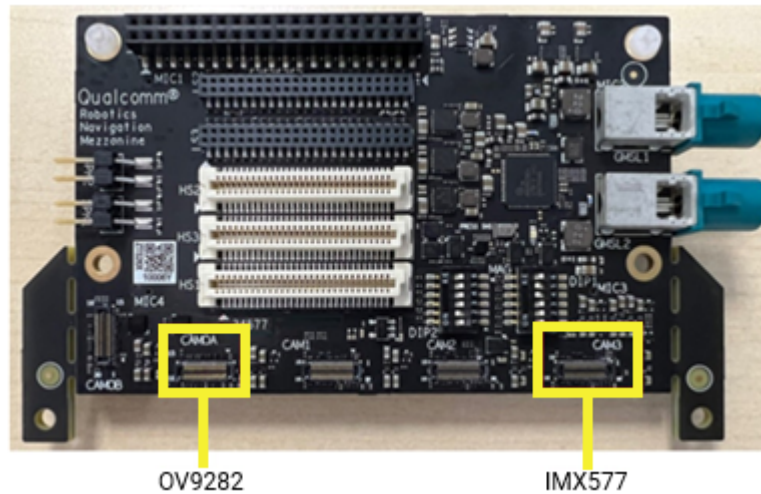
3.1 Set up the camera

QCS6490

3.1.1 Connect RB3G2 MIPI CSI on vision mezzanine board

OV9282 and IMX577 are the default camera sensors.

Connect the OV9282 module to CAM0A and the IMX577 module to CAM3 port as shown:



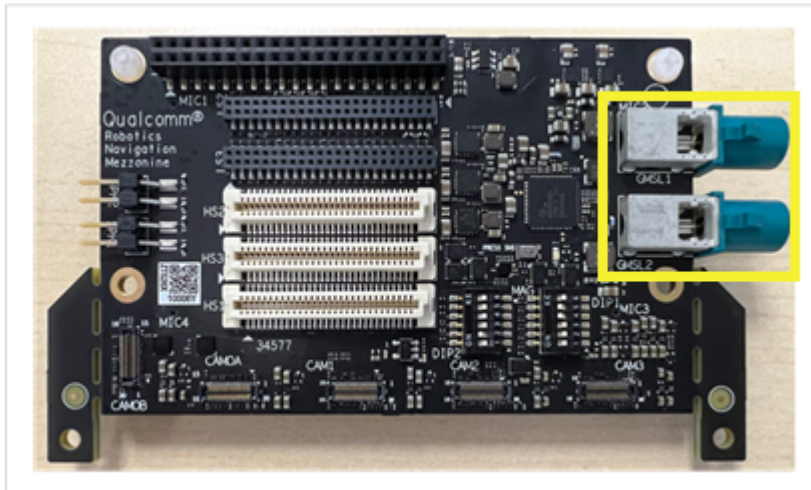
3.1.2 Set SW2300 DIP switch on interposer board

To use the CAM3 slot of the vision mezzanine board, switch 1 of the SW2300 DIP switch on the interposer board must be set to OFF. This switch is set to OFF by default. The CAM0A slot of the vision mezzanine board works without any DIP switch setting.



3.1.3 Connect RB3G2 GMSL cameras on vision mezzanine board

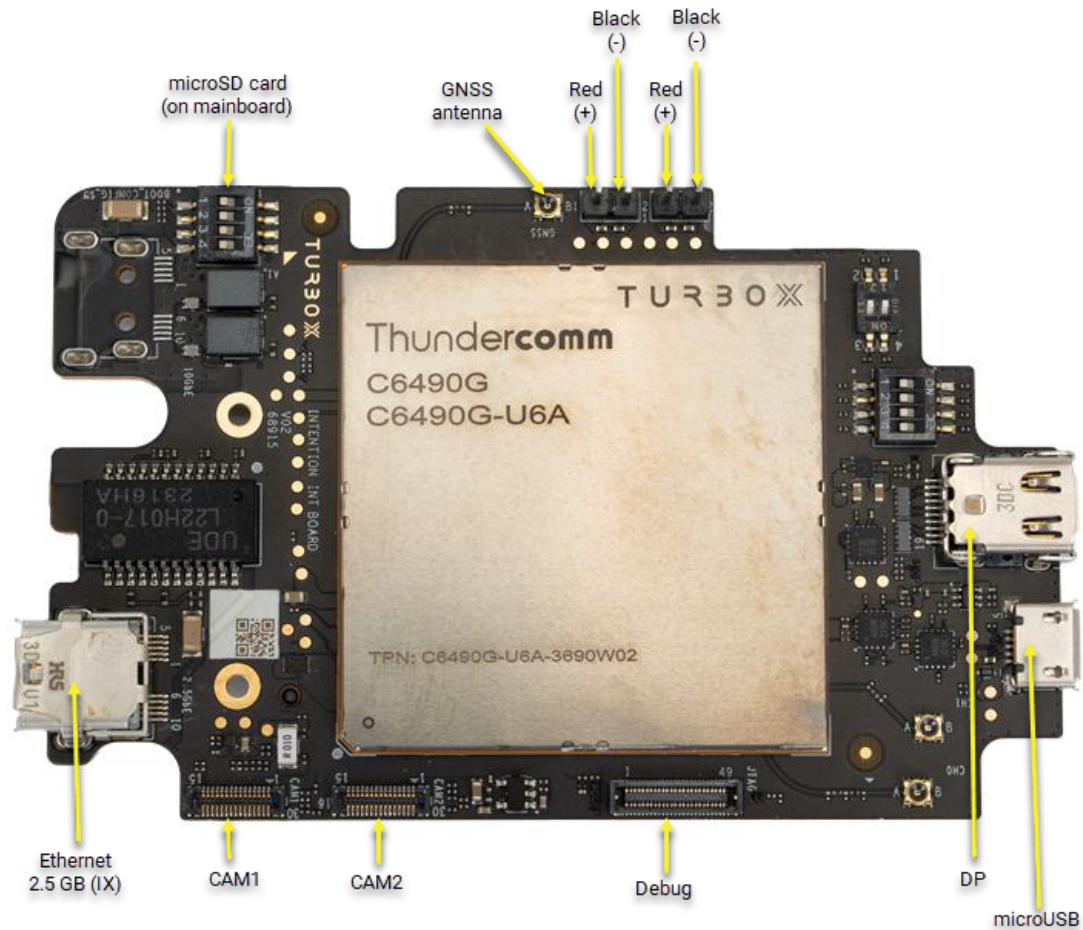
There are two GMSL slots (GMSL1 and GMSL2) on the mezzanine board of Qualcomm's RB3G2 Vision Kit platform. Two GMSL cameras can be connected to these slots.



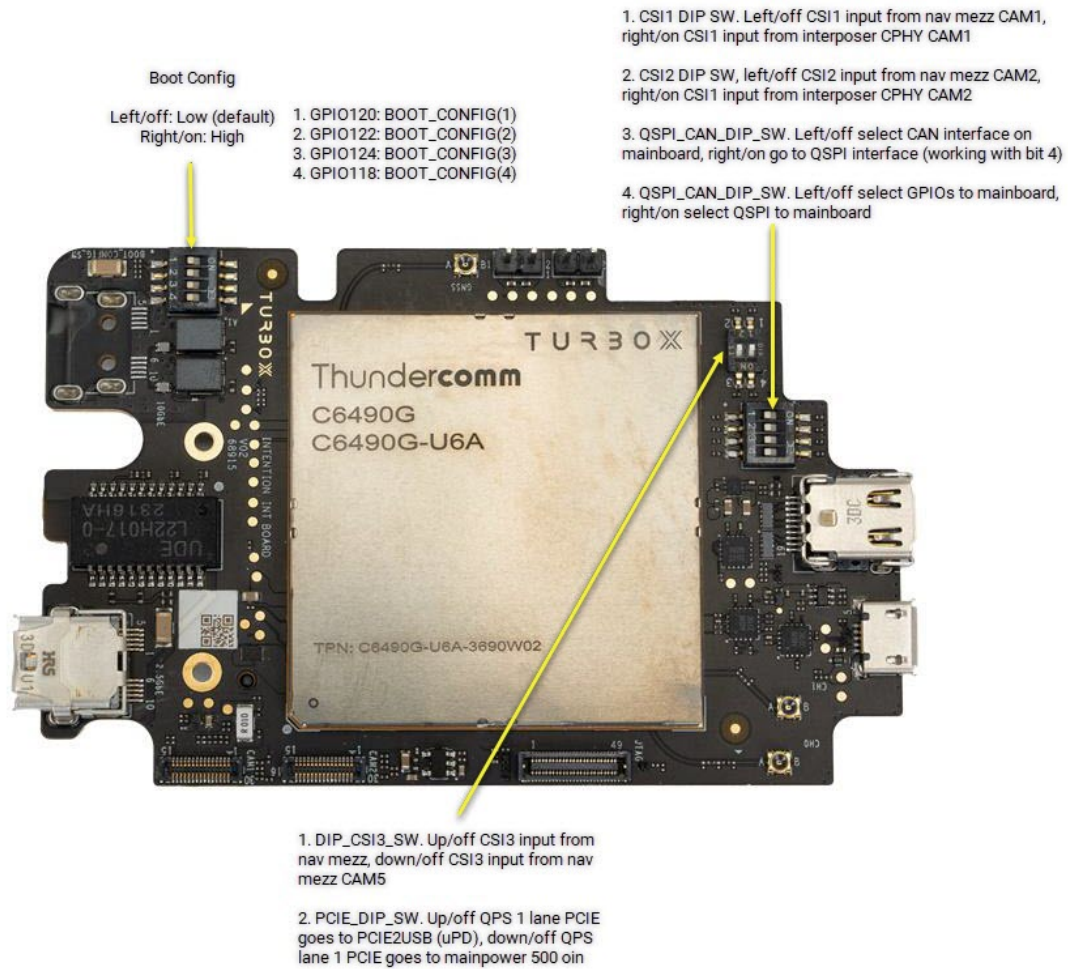
The GMSL1 slot can be used without any hardware setting. The CAM3 CSI slot and GMSL2 slot on the vision mezzanine board share the same CSI. To use the GMSL2 slot, switch 1 of the SW2300DIP switch on the interposer board must be set to ON. This switch is set to OFF by default. See [SW2300 Dip switch on interposer board](#).

3.1.4 Connect RB3G2 MIPI CSI cameras on interposer board

Connect OV9282 module to CAM1 and IMX577 module to CAM2 on the interposer board.



Set CSI1_DIP_SW and CSI2_DIP_SW to ON to use CAM1/CAM2 on the interposer board as shown in the following image. The switch is set OFF by default.



3.1.5 Supported resolutions and features

The following table shows the supported resolutions each camera module.

Resolution	Aspect Ratio	IMX577 (CAM3)	OV9282 (CAM0A)	AR0231 (GMSL)
4000 x 3000	4:3	Yes	No	No
3840 x 2160	16:9	Yes	No	No
2976 x 2976	1:1	Yes	No	No
2592 x 1940	4:3	Yes	No	No
2048 x 1536	4:3	Yes	No	No
1920 x 1440	4:3	Yes	No	No
1928 x 1208	16:10	Yes	No	Yes
1920 x 1080	16:9	Yes	No	Yes
1440 x 1080	4:3	Yes	No	Yes
1280 x 720	16:9	Yes	Yes	Yes
1024 x 768	4:3	Yes	Yes	Yes
800 x 600	4:3	Yes	Yes	Yes
640 x 480	4:3	Yes	Yes	Yes

Resolution	Aspect Ratio	IMX577 (CAM3)	OV9282 (CAM0A)	AR0231 (GMSL)
640 x 360	16:9	Yes	Yes	Yes
320 x 240	4:3	Yes	Yes	Yes

The following table shows the supported features of each camera module.

Feature	IMX577 (CAM3)	OV9282 (CAM0A)	AR0231 (GMSL)
SHDR	Yes	No	No
LDC	Yes	No	No
EIS	Yes	No	No

QCS9075

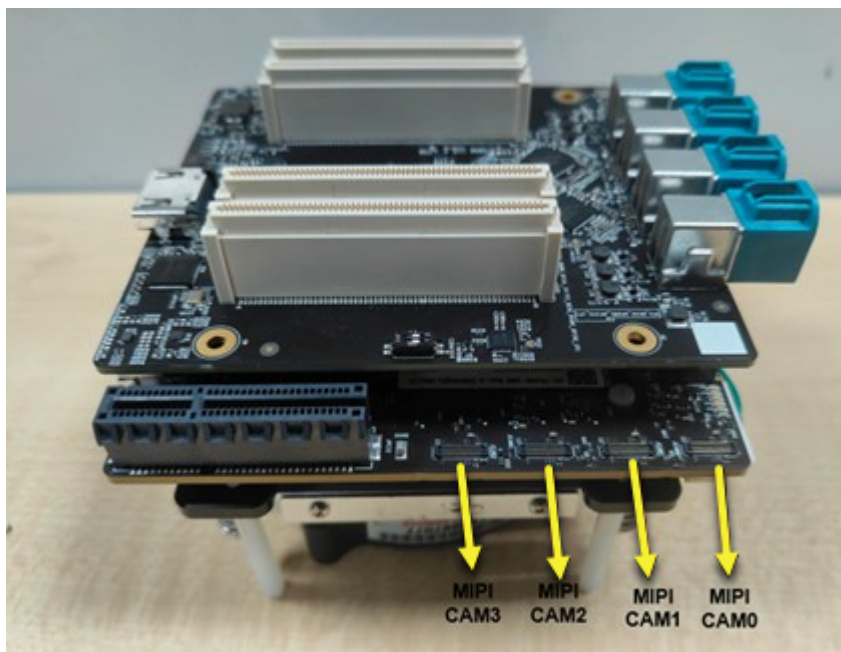
This section explains how to connect camera sensors on the QCS9075 reference platform (RB8).

3.1.6 Connect MIPI CSI cameras on RB8

RB8 will be offered in one kit version called the core kit. This core kit supports only MIPI CSI cameras. The GMSL cameras are supported using a separate add-on GMSL mezzanine board that needs to be connected to core kit.

In the RB8 (core kit + GMSL mezzanine) hardware, there are four MIPI CSI (C/D-PHY) connectors present on the RB8 core kit/main board and four GMSL ports (0 to 3) present on the GMSL mezzanine board.

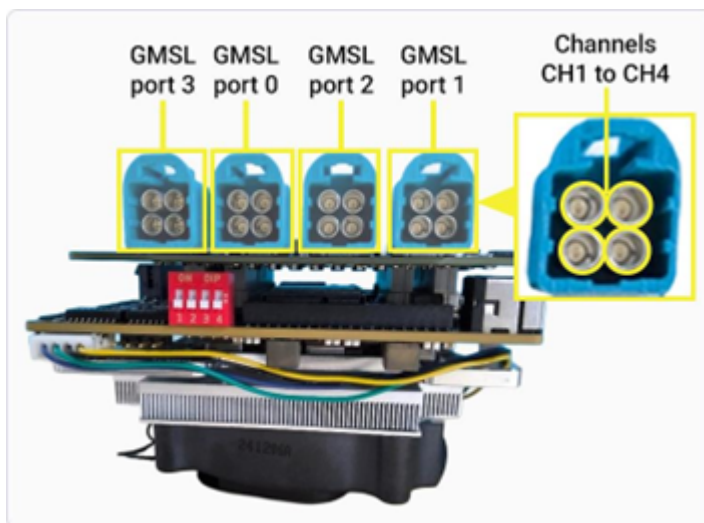
The following diagram shows the MIPI connectors on the RB8 core kit/main board.



3.1.7 Connect GMSL cameras on RB8

There are four GMSL ports (0 to 3) present on the GMSL mezzanine board. Each GMSL port connects with a MAX96724 quad GMSL deserializer. Each deserializer is connected to one CSI.

The following diagram shows the GMSL ports on the RB8 device. Note that the GMSL port numbering is not in sequence - It is defined based on the CSI index to which a GMSL port is connected.



Note:

In the current release three GMSL ports (0, 2, and 3) are enabled to connect GMSL cameras. Port 1 is not enabled.

- Each GMSL port contains four slots (1 to 4, also referred to as channels) and supports connection of four GMSL cameras on a single port.
- The current release supports OX03f10 Bayer GMSL and OX03f10 YUV GMSL cameras.
- Port-0 supports OX03f10 Bayer GMSL cameras. Port-2 and Port-3 support OX03f10 YUV GMSL cameras. Thus, you can connect four OX03F10 Bayer GMSL cameras on GMSL Port-0, four OX03F10 YUV GMSL cameras on GMSL Port-2 and four OX03F10 YUV GMSL cameras on GMSL Port-3.

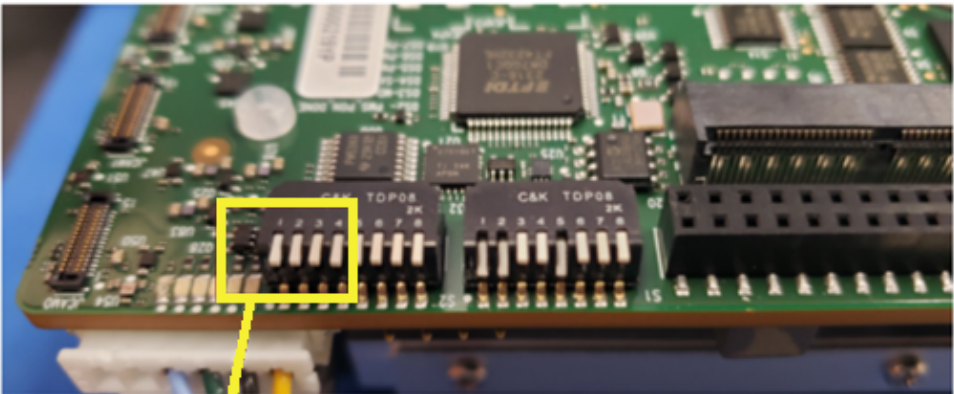
Note:

In the current release, OX03f10 bayer GMSL cameras connected to Port-0 are not stable on the RB8 hardware due to CSI errors. This will be fixed in the next release.

3.1.8 Set DIP switches

QCS9075 has four CSI PHYs. Each CSI PHY is routed to connect to either a MIPI camera port or a GMSL camera port controlled using DIP switch present on the core kit/main board. The switch is different for EVT and DVT/PVT hardware versions:

- DIP switch setting for RB8 EVT hardware: The following diagram shows DIP switches present on the core kit/main board.



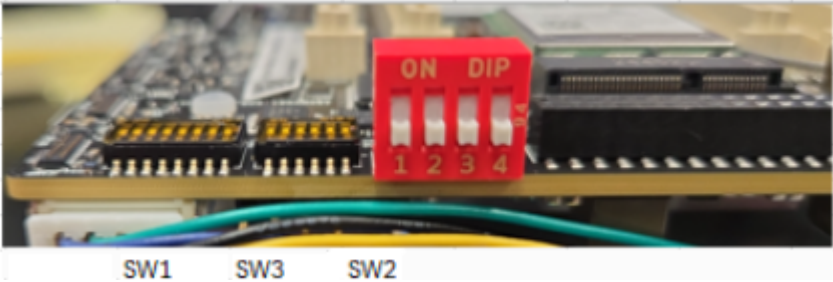
Switch 2 Switch 1

Pins 1, 2, 3, and 4 of SW2 should be moved up to enable MIPI and moved down to enable GMSL sensors

The following table explains the required DIP switch SW2 settings needed to use MIPI and GMSL camera ports.

Switch	OFF (default from factory)	ON	Connection when on	Connection when off (default from factory)
SW2 - 1	HIGH	LOW	CSI0 connected to GMSL mezzanine	CSI0 connected to main board
SW2 - 2	HIGH	LOW	CSI1 connected to GMSL mezzanine	CSI1 connected to main board
SW2 - 3	HIGH	LOW	CSI2 connected to GMSL mezzanine	CSI2 connected to main board
SW2 - 4	HIGH	LOW	CSI3 connected to GMSL mezzanine	CSI3 connected to main board

- DIP switch setting for RB8 DVT/PVT hardware: The following diagram shows DIP switches present on the core kit/main board.



The following table explains the required DIP switch SW1 settings needed to use MIPI and GMSL camera ports. On the DIP switch, it is written “ON” to know the switch direction to enable.

Switch	Connection when on	Connection when off (default from factory)
SW1 - 5	CSI0 connected to GMSL mezzanine	CSI0 connected to main board
SW1 - 6	CSI1 connected to GMSL mezzanine	CSI1 connected to main board
SW1 - 7	CSI2 connected to GMSL mezzanine	CSI2 connected to main board
SW1 - 8	CSI3 connected to GMSL mezzanine	CSI3 connected to main board

3.1.9 Supported resolutions and features

The following table shows the supported resolutions each camera module on the RB8 platform.

Resolution	Aspect ratio	0X3F10 Bayer GMSL	0X3F10 YUV GMSL	OV9282 (CAM0A)
1920 x 1536	5:4	No	Yes	No
1920 x 1440	4:3	No	Yes	No
1928 x 1208	16:10	No	Yes	No
1920 x 1080	16:9	No	Yes	No
1824 x 1536	19:16	Yes	Yes	No
1440 x 1080	4:3	Yes	Yes	No
1280 x 720	16:9	Yes	Yes	Yes
1024 x 768	4:3	Yes	Yes	Yes
800 x 600	4:3	Yes	Yes	Yes
640 x 480	4:3	Yes	Yes	Yes
640 x 360	16:9	Yes	Yes	Yes
320 x 240	4:3	Yes	Yes	Yes

Advanced features such as SHDR, LDC, and EIS are not supported on QCS9075.

3.1.10 Concurrent camera support on RB8

The following tables explain the concurrent camera use cases which are supported in the current release.

- RB8 MIPI sensor: The software supports the OV9282 MIPI camera. The following table describes MIPI camera support on the RB8 platform.

MIPI camera	Supported resolution and FPS	CAM0	CAM1	CAM2	CAM3	Notes
OV9282	1280x720, 30 FPS	Supported	Supported	Supported	Supported	Any camera can operate independently. A maximum of two cameras can operate concurrently with real-time IFE processing. All four cameras can operate concurrently with the offline IFE feature.

- RB8 GMSL sensor: The software supports OX03F10 Bayer and OX03F10 YUV GMSL cameras. The following table describes GMSL camera support on the RB8 platform.

GMSL port	Supported resolution and FPS	GMSL sensor	Note
Port-0	1824x1536, 30 FPS	3F10 GMSL Bayer	Four OX03F10 BayerGMSL cameras can be connected on GMSL Port-0. Any one camera can operate independently. All four cameras on a single port can operate concurrently using the Per Port Group feature. Note: In the current release, OX03f10 Bayer GMSL cameras connected to Port-0 are not stable on the RB8 hardware due to CSI errors. This will be fixed in the next release.

GMSL port	Supported resolution and FPS	GMSL sensor	Note
Port-2	1920x1536,30 FPS	3F10 GMSL YUV	Four OX03F10 YUV GMSL cameras can be connected on GMSL Port-2. Any one camera can operate independently. All four cameras on a single port can operate concurrently using the Per Port Group feature.
Port-3	1920x1536,30 FPS	3F10 GMSL YUV	Four OX03F10 YUV GMSL cameras can be connected on GMSL Port-3. Any one camera can operate independently. All four cameras on a single port can operate concurrently using the Per Port Group feature.
GMSL concurrency with Port-2 + Port-3 Note: Port-0 is not considered for concurrency here as this port is not stable due to CSI errors (a known issue on RB8 for this release). This will be fixed in the next release.	1920x1536,30 FPS	3F10 GMSL YUV	Eight OX03F10 YUV GMSL cameras can be connected, four on Port-2 and four on Port-3. Any one camera from each port can operate independently. All eight cameras can operate concurrently using the Per Port Group feature.

- RB8 GMSL sensor + MIPI sensor concurrency: The following GMSL sensor + MIPI sensor concurrency is supported on the RB8 platform.

	CAM0	CAM1	GMSL Port-2	GMSL Port-3	Notes
Camera Sensor and supported resolution	OV9282 1280x720,30 FPS	OV9282 1280x720,30 FPS	OX03F10 YUV GMSL 1920x1080,30 FPS	OX03F10 YUV GMSL 1920x1080,30 FPS	Any camera can operate independently. All four cameras operate concurrently.

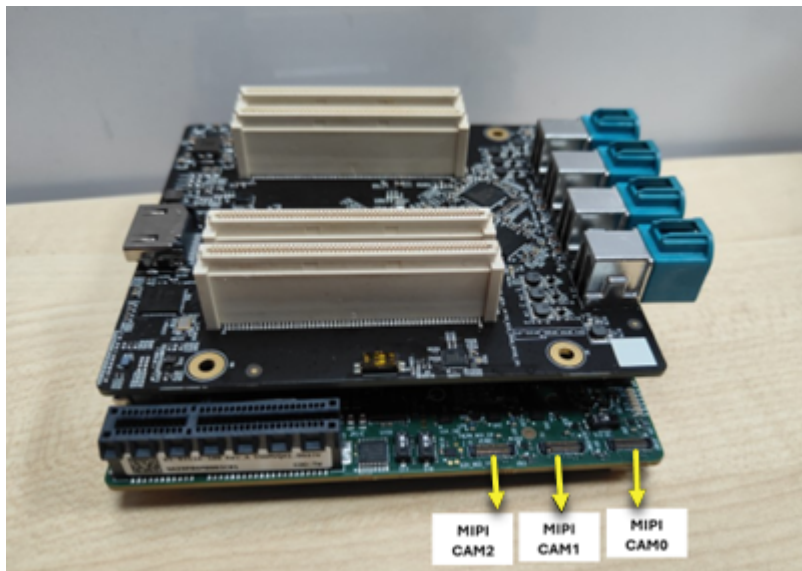
QCS8275

This section explains how to connect camera sensors to the QCS8275 reference platform (RB4).

3.1.11 Connect MIPI CSI camera module on RB4

RB4 is offered in the “core” kit. This core kit supports MIPI CSI cameras. GMSL cameras are supported using a separate add-on GMSL mezzanine board that must be connected to the core kit.

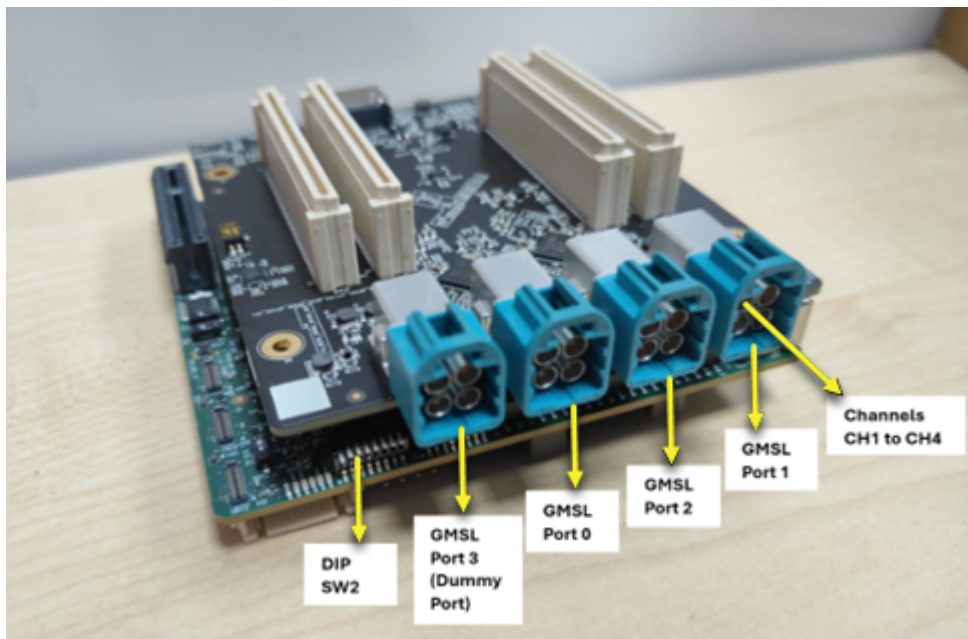
The following diagram shows the MIPI connectors on the RB4 core kit/main board.



3.1.12 Connect GMSL camera module on RB4

There are four GMSL ports (0 to 3) present on the GMSL mezzanine board. Port 3 is a dummy port. GMSL ports 0, 1, and 2 each connect with a MAX96724 quad GMSL deserializer. Each deserializer is connected to one CSI.

The following diagram shows the GMSL ports on the RB4 hardware. Note that GMSL port numbering is not in sequence. It is defined based on the CSI index to which the GMSL port is connected.



Note:

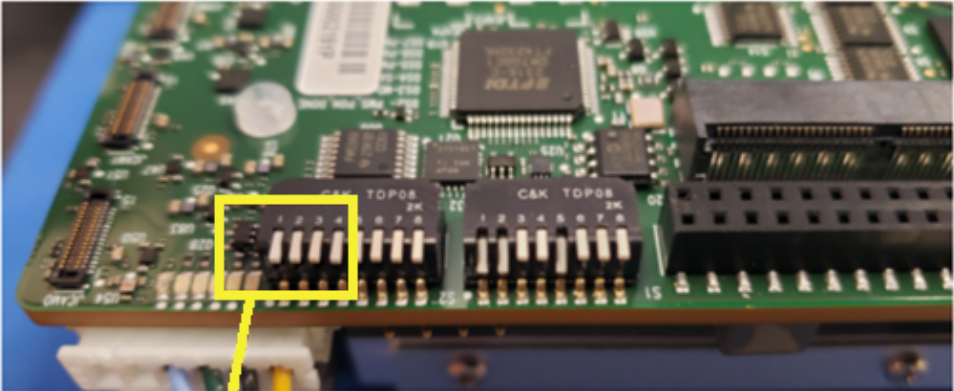
In the current release, two GMSL ports (0 and 2) are supported to connect GMSL cameras. Port 1 is not enabled due to a hardware issue. Port 3 is a dummy port.

- Each GMSL port contains four slots (1 to 4, also referred to as channels) and supports connection to four GMSL cameras on single port.
- The current release supports OX03f10 bayer GMSL and OX03f10 YUV GMSL cameras.
- Port-0 supports OX03f10 Bayer GMSL cameras. Port-2 supports OX03f10 YUV GMSL cameras. Thus, you can connect four OX03F10 bayer GMSL cameras on GMSL Port-0, and four OX03F10 YUV GMSL cameras on GMSL Port-2.

3.1.13 Set DIP switches

QCS8275 has three CSI PHYs. Each CSI PHY is routed to connect to either a MIPI camera port or a GMSL camera port controlled with a DIP switch present on the core kit/main board. The switch is different for EVT and DVT/PVT hardware versions.

- DIP switch setting on RB4 EVT hardware



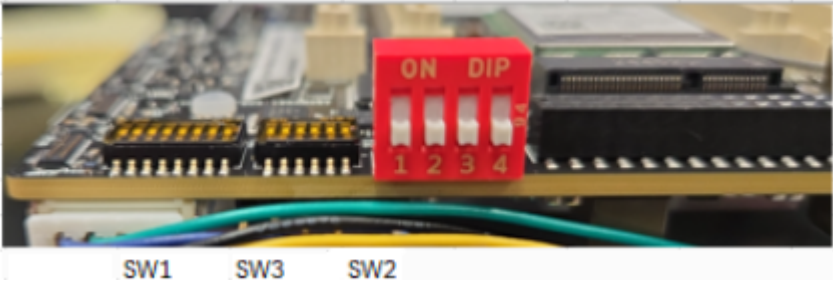
Switch 2 Switch 1

Pins 1, 2, and 3 of SW2 should be moved up to enable MIPI and moved down to enable GMSL sensors

The following table explains the required DIP switch SW2 settings needed to use MIPI cameras and GMSL camera ports.

Switch	OFF (default from factory)	ON	Connection when on	Connection when off (default from factory)
SW2 - 1	HIGH	LOW	CSI0 connected to GMSL mezzanine	CSI0 connected to main board
SW2 - 2	HIGH	LOW	CSI1 connected to GMSL mezzanine	CSI1 connected to main board
SW2 - 3	HIGH	LOW	CSI2 connected to GMSL mezzanine	CSI2 connected to main board

- DIP switch setting on RB4 DVT/PVT hardware



The following table explains the required DIP switch SW1 settings needed to use MIPI and GMSL camera ports. On the DIP switch, it is written “ON” to know which direction to enable.

Switch	Connection when ON	Connection when OFF (default from factory)
SW1-5	CSI0 connected to GMSL mezzanine	CSI0 connected to main board
SW1-6	CSI1 connected to GMSL mezzanine	CSI1 connected to main board
SW1-7	CSI2 connected to GMSL mezzanine	CSI2 connected to main board

3.1.14 Supported resolutions and features

The following table shows the supported resolutions each camera module on the RB4 platform.

Resolution	Aspect ratio	0X3F10 Bayer GMSL	0X3F10 YUV GMSL	OV9282 (CAM0A)
1920 x 1536	5:4	No	Yes	No
1920 x 1440	4:3	No	Yes	No
1928 x 1208	16:10	No	Yes	No
1920 x 1080	16:9	No	Yes	No
1824 x 1536	19:16	Yes	Yes	No
1440 x 1080	4:3	Yes	Yes	No
1280 x 720	16:9	Yes	Yes	Yes
1024 x 768	4:3	Yes	Yes	Yes
800 x 600	4:3	Yes	Yes	Yes
640 x 480	4:3	Yes	Yes	Yes
640 x 360	16:9	Yes	Yes	Yes
320 x 240	4:3	Yes	Yes	Yes

Advanced features such as SHDR, LDC, and EIS are not supported on QCS8275.

3.1.15 Concurrent camera support on RB4

The following tables explain the concurrent camera use cases which are supported in the current release.

- RB4 MIPI sensor: The software supports the OV9282 MIPI camera. The following table describes MIPI camera support on the RB4 platform.

MIPI camera	Supported resolution and FPS	CAM0	CAM1	CAM2	Notes
OV9282	1280x720, 30 FPS	Supported	Supported	Supported	Any camera can operate independently. A maximum of two cameras can operate concurrently with real-time IFE processing. All three cameras can operate concurrently using the Offline IFE feature.

- RB4 GMSL sensor: The software supports OX03F10 bayer and OX03F10 YUV GMSL cameras. The following table describes GMSL camera support on the RB4 platform.

GMSL port	Supported resolution and FPS	GMSL sensor	Note
Port-0	1824x1536, 30 FPS	3F10 GMSL Bayer	Four OX03F10 BayerGMSL cameras can be connected on GMSL Port-0. Any one camera can operate independently. All four cameras on a single port can operate concurrently using the Per Port Group feature. Note: In the current release, running four OX03f10 bayer GMSL cameras connected to Port-0 is not stable on the RB4 hardware. This will be fixed in the next release.

GMSL port	Supported resolution and FPS	GMSL sensor	Note
Port-2	1920x1536,30 FPS	3F10 GMSL YUV	Four OX03F10 YUV GMSL cameras can be connected on GMSL Port-2. Any one camera can operate independently. All four cameras on a single port can operate concurrently using the Per Port Group feature. Note: In the current release, running four OX03f10 bayer GMSL cameras connected to Port-2 is not stable on the RB4 hardware. This will be fixed in the next release.
GMSL concurrency with Port-0 + Port-2	Port-0: 1824x1536, 30 FPS Port-2: 1920x1536,30 FPS	Port-0: 3F10 GMSL bayer Port-2: 3F10 GMSL YUV	Four OX03F10 bayer GMSL cameras can be connected on Port-0, and four OX03F10 YUV GMSL cameras can be connected on Port-2. Any one camera from each port can operate independently. All eight cameras can operate concurrently using the Per Port Group feature. Note: In the current release, this concurrency is not stable on the RB4 hardware. This will be fixed in the next release.

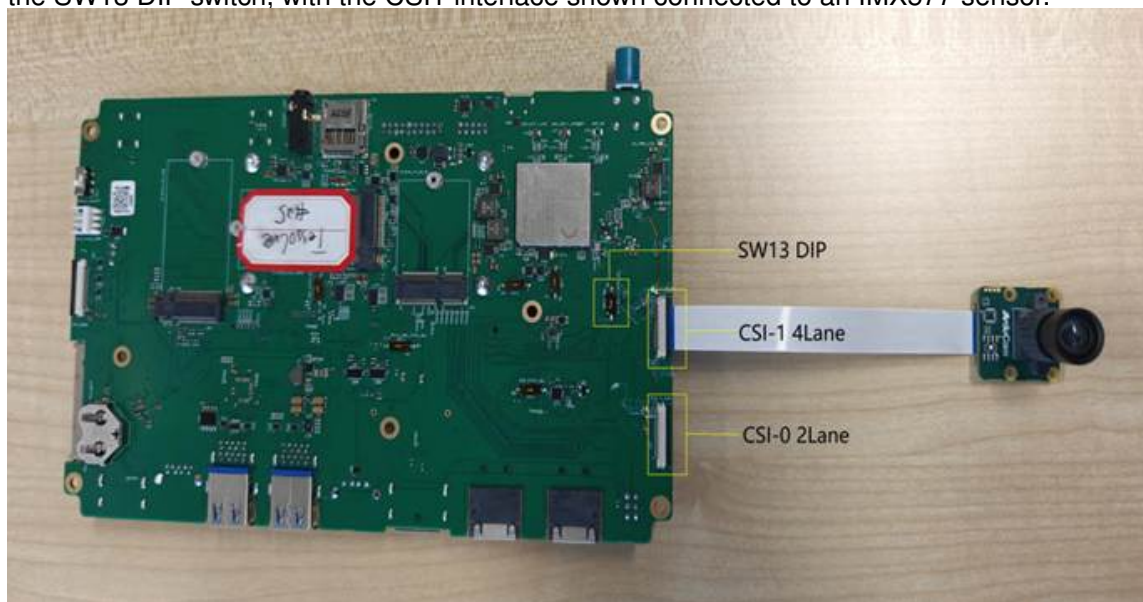
- RB4 GMSL sensor + MIPI sensor concurrency: The following GMSL sensor + MIPI sensor concurrency is supported on the RB4 platform.

	GMSL Port-0 CAM0	CAM1	GMSL Port-2	Notes
Camera Sensor and supported resolution	OX03F10 bayer GMSL 1824x1536, 30 FPS	OV9282 MIPI camera 1280x720,30 FPS	OX03F10 YUV GMSL 1920x1080,30 FPS	Any camera can operate independently. All three cameras operate concurrently.

	GMSL Port-0 CAM0	CAM1	GMSL Port-2	Notes
	OV9282 MIPI camera 1280x720,30 FPS	OV9282 MIPI camera 1280x720,30 FPS	OX03F10 YUV GMSL 1920x1080,30 FPS	Any camera can operate independently. All three cameras operate concurrently.

IQ615

The IQ615 EVK board provides support for MIPI CSI cameras. GMSL cameras are supported through a dedicated port. The image below shows the MIPI and GMSL connectors on the IQ615 EVK board. The IQ615 EVK doesn't include a default sensor connection. It provides two 22-pin CSI interfaces: CSI0 (2-lane) and CSI1 (4-lane). The CSI1 interface supports MIPI-to-GMSL path switching via the SW13 DIP switch. To enable the CSI1 interface, DIP switch SW13 must be set to ON. By default, SW13 is set to OFF, which configures the interface for GMSL. The CSI0 interface can be used directly without any additional hardware configuration. The following image shows the locations of the CSI0 and CSI1 connectors and the SW13 DIP switch, with the CSI1 interface shown connected to an IMX577 sensor.



The location of the GMSL interface is shown in the following image. The GMSL slot can be used directly without any additional hardware configuration, as the SW13 DIP switch is set to OFF by default.



Note:

GMSL cameras are currently not enabled on the IQ615 EVK board.

3.1.16 IQ615 EVK board switch configuration

Carrier board switch function	Switch configuration	Comment
GMSL deserializer Select switches to enable MIPI for CSI1 port	SW13 - OFF	From GMSL deserializer to CSI1 port
22 pin camera connector Select switches to enable MIPI for CSI port	SW13 - ON	From 22-pin camera connector to CSI1 port

3.2 Choose the stream API

Qualcomm Linux supports following APIs for camera.

- [GStreamer API](#)
GStreamer is an open-source multimedia framework. Qualcomm provides a GStreamer plugin (qtiqmmfsrc) that allows developers to control the camera subsystem in applications. See [Qualcomm GStreamer plugins](#) for more information.
- [V4L2 API](#)
V4L2 is a framework within the Linux kernel that provides support for video capture, video output, and other multimedia devices. Developers can operate the camera using the V4L2 API. The V4L2 API, which uses the CAMSS driver, is suitable for developers who only need to obtain raw images from the camera.

3.3 Stream camera with the GStreamer API

gst-launch-1.0 is a command-line GStreamer utility used to build and run a GStreamer pipeline. The pipeline is specified as a collection of elements with properties separated by exclamation marks (!).

3.3.1 Prerequisites

To use [gst-launch-1.0](#) and GStreamer plugins, flash the Config2 image. For overlay configuration details, see [Overlay Configurations](#).

Note:

Connect to the device console using SSH. See [How To SSH?](#) for instructions.

Run the following command in an SSH terminal:

```
# mount -o rw,remount /usr
```

If a GStreamer command was executed before enabling the camera overlay, the GStreamer registry must be cleared after enabling the camera overlay. Run the following command to clear the registry:

```
rm ~/.cache/gstreamer-1.0/registry.aarch64.bin
```

3.3.2 Run camera use cases

QCS6490

Note:

Ensure that MIPI cameras are connected to CSI slots. The OV9282 MIPI camera should be connected to the CAM 0A slot and the IMX577 MPI camera should be connected to the CAM3 slot.

Single camera stream start

1. Run the following command in the device terminal:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! 'video/x-raw,
format=NV12,\
width=1280,height=720,framerate=30/1' ! fakesink
```

2. This command starts the camera with 720p at 30 FPS configuration. The frame coming from the camera sensor is thrown away by fakesink. If the gst pipeline status is changed to “PLAYING” as shown below, this indicates that the camera is running. Since this command dumps camera frames to fakesink, nothing will be saved on the device.

```
gbm_create_device(187): Info: backend name is: msm_drm
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
```

To stop the camera, press **CTRL+C**.

Video encoding

1. Run the following command in the device terminal:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
mux_avc.mp4
```

This command starts the camera with 720p at 30 FPS configuration and saves it as a video file after h264 video encoding. If the gst pipeline status is changed to “PLAYING”, this indicates the camera is running.

To stop the camera, press **CTRL+C**.

Note:

Back-to-back execution of this use case is not supported in the RC2 release and will be addressed in an upcoming release.

2. `/opt/mux_avc.mp4` is generated on the device. The recorded content can be pulled from the device by running the following scp command on the host PC:

```
$ scp -r root@[ip-addr]:/opt/mux_avc.mp4 .
```

Video encoding and snapshot

1. Run the following command in the device terminal:

Note:

gst-pipeline-app is currently not enabled from GStreamer.

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
mux_avc.mp4 \
camsrc.image_1 ! "image/jpeg,width=1280,height=720,framerate=30/1" \
! multifilesink location=/opt/frame%d.jpg async=false sync=true enable-
last-sample=false
```

2. Press **Enter**. This command will print the following menu and wait for user input.

```
##### MENU #####
#####

===== Pipeline
Controls=====
(0) NULL: Set the pipeline into NULL state
(1) READY: Set the pipeline into READY state
(2) PAUSED: Set the pipeline into PAUSED state
(3) PLAYING: Set the pipeline into PLAYING state

=====
Other=====
(p) Plugin Mode: Choose a plugin which to control
(q) Quit : Exit the application

Choose an option:
```

3. Use the following menu steps to take a snapshot while recording video.

```
(1) ready -> (3) Playing -> (p)Plugin Mode : Select (8)camerasrc ->
(37) capture-image -> (1): still - Snapshot ->(1) Snapshot count (
'guint' value for arg1)
```

4. To stop the camera, press **Enter**, press **b** (back), and then press **q** (quit). The recorded video file and snapshot images are saved in `/opt/`. The recorded content can be pulled from the device by running the following scp command on the host PC:

```
$ scp -r root@[ip-addr]:/opt/<file name> .
```

MIPI multi-camera streaming

QCS6490 has five CSI PHYs to which you can connect five MIPI cameras.

QCS6490 has three IFEs and two IFE Lites. This means you can run three cameras with IFEs and two cameras with IFE Lites. IFE Lites only support raw dumps from the sensor. You can connect mono cameras to the IFE Lites, dump raw frames, and then convert the output to YUV frames using a CHI node.

To validate five camera concurrency on the RB3 platform:

1. Connect MIPI cameras as described in the following table.

Slot	Camera sensor
CAM 0A	OV9282 mono camera
CAM 0B	OV9282 mono camera
CAM1	IMX577 bayer camera
CAM2	IMX577 bayer camera
CAM3	IMX577 bayer camera

2. Run the following GST commands to run all five cameras concurrently.

Note:

For OV9282 mono cameras, a separate camera pipeline (RealTimeMonoToYUV) is used. This pipeline uses IFE Lites to dump the raw frame and then convert it to a YUV frame using a CHI node. Use the GST parameter `ife-direct-stream=1` to use this camera pipeline. Since it runs with an IFE Lite, 2A is not applied on these cameras.

Note:

The QCS6490 maximum encode capability is **4K@30**. Thus, the total encode processing from all five cameras has to be under the **4k@30** limit.

- Cam 0: IMX577

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! video/x-raw,format=Nv12,width=1920,height=1080,\
framerate=30/1 ! v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_bitrate=5000000,\
video_bitrate_mode=0;" ! queue ! h264parse ! mp4mux ! queue ! filesink location=/opt/Cam_0_1080p.mp4
```

- Cam 1: IMX577

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=1 ! video/x-raw,format=Nv12,width=1920,height=1080,\
framerate=30/1 ! v4l2h264enc capture-io-mode=4 output-io-
```

```
mode=5 extra-controls="controls,video_bitrate=5000000,\
video_bitrate_mode=0;" ! queue ! h264parse ! mp4mux ! queue
! filesink location=/opt/Cam_1_1080p.mp4
```

- Cam 8: IMX577

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=8 ! video/x-
raw,format=Nv12,width=1920,height=1080,\
framerate=30/1 ! v4l2h264enc capture-io-mode=4 output-io-
mode=5 extra-controls="controls,video_bitrate=5000000,\
video_bitrate_mode=0;" ! queue ! h264parse ! mp4mux ! queue
! filesink location=/opt/Cam_8_1080p.mp4
```

- Cam 2: OV9282

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=2 ife-
direct-stream=1 ! video/x-raw,format=Nv12,width=1280,
height=720,\
framerate=30/1 ! v4l2h264enc capture-io-mode=4 output-io-
mode=5 extra-controls="controls,video_bitrate=5000000,\
video_bitrate_mode=0;" ! queue ! h264parse ! mp4mux ! queue
! filesink location=/opt/Cam_2_720p.mp4
```

- Cam 3 : OV9282

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=3 ife-
direct-stream=1 ! video/x-raw,format=Nv12,width=1280,
height=720,\
framerate=30/1 ! v4l2h264enc capture-io-mode=4 output-io-
mode=5 extra-controls="controls,video_bitrate=5000000,\
video_bitrate_mode=0;" ! queue ! h264parse ! mp4mux ! queue
! filesink location=/opt/Cam_3_720p.mp4
```

QCS9075

Note:

Ensure that the camera (MIPI or GMSL) sensor is connected to the RB8 device. You can connect OV9282 MIPI cameras on CSI slots. Connect OX03f10 Bayer GMSL cameras to GMSL Port-0 and OX03f10 YUV GMSL cameras to Port-2 and Port-3.

Note:

There are random CSI errors on GMSL Port-0 that stop Bayer GMSL camera streaming. This issue will be fixed in the next GA release.

Single camera stream start

Run the following command in the device terminal:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! 'video/x-raw,
format=NV12,\
width=1280,height=720,framerate=30/1' ! fakesink
```

The following command starts the camera with 720p at 30 FPS configuration. The frame coming from the camera sensor is thrown away by fakesink. If the gst pipeline status is changed to “PLAYING” as shown below, this indicates that the camera is running. Since this command dumps camera frames to fakesink, nothing will be saved on the device.

```
gbm_create_device(187): Info: backend name is: msm_drm
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
```

To stop the camera, press **CTRL+C**.

Video encoding

1. Run the following command in the device terminal:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
mux_avc.mp4
```

This command starts the camera with 720p at 30 FPS configuration and saves it as a video file after h264 video encoding. If the gst pipeline status is changed to “PLAYING”, this indicates the camera is running.

To stop the camera, press **CTRL+C**.

2. /opt/mux_avc.mp4 is generated on the device. The recorded content can be pulled from the device by running the following scp command on the host PC:

```
$ scp -r root@[ip-addr]:/opt/mux_avc. mp4 .
```

Video encoding and snapshot

QCS9075 doesn't have a hardware JPEG encoder. The snapshot use case works using a software JPEG encoder algorithm that runs on the CPU.

1. Run the following command in the device terminal:

Note:

gst-pipeline-app is currently not enabled from GStreamer.

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc camera=0 ! video/x-raw,
format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! queue !
v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate_mode=0,video_
bitrate=6000000;" ! queue ! \
h264parse ! mp4mux ! queue ! filesink location="/opt/mux_720_avc.mp4"
camsrc.image_1 ! "image/jpeg,\
width=1280,height=720,framerate=30/1" ! multifilesink location=/opt/
frame%d.jpg async=false sync=true \
enable-last-sample=false
```

2. Press **Enter**. This command will print the following menu and wait for user input.

```
##### MENU #####
### #####

===== Pipeline
Controls=====
(0)      NULL: Set the pipeline into NULL state
(1)      READY: Set the pipeline into READY state
(2)      PAUSED: Set the pipeline into PAUSED state
(3)      PLAYING: Set the pipeline into PLAYING state
=====
Other=====
(p)      Plugin Mode: Choose a plugin which to control
(q)      Quit : Exit the application

Choose an option:
```

3. Use the following menu steps to take a snapshot while recording video.

```
(1) ready -> (3) Playing -> (p)Plugin Mode : Select (11)camerasrc
->
```

```
(38) capture-image -> (1): still - Snapshot ->(1) Snapshot count (
'guint' value for arg1)
```

4. To stop the camera, press **Enter**, press **b** (back), and then press **q** (quit). The recorded video file and snapshot images are saved in `/opt/`. The recorded content can be pulled from the device by running the following scp command on the host PC:

```
$ scp -r root@[ip-addr]:/opt/<file name> .
```

MIPI multi-camera streaming

There are four MIPI CSI slots present on the RB8 main board. Connect OV9282 cameras to MIPI CSI ports. You can run any two OV9282 cameras concurrently out of four cameras.

Run the following GST command in two different terminals with camera = 0, 1 to validate MIPI camera concurrency. Use different file names when saving the encoded file of each camera.

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls, video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/mux_
avc.mp4
```

You can also run all four MIPI cameras concurrently using the offline IFE feature. Refer to [Support multi-camera using offline IFE](#) for information on four camera validation.

GMSL multi-camera streaming

A single GMSL port contains four slots, allowing connection of four GMSL cameras to a single CSI using different virtual channel IDs. The four cameras connected on one CSI are treated as group. A group of cameras can be run concurrently using the [per port grouping feature](#).

QCS8275

Note:

Ensure that the camera (MIPI or GMSL) sensor is connected to the RB4 device. You can connect OV9282 MIPI cameras on CSI slots. Connect OX03f10 Bayer GMSL cameras to GMSL Port-0 and OX03f10 YUV GMSL cameras to GMSL

Port-2 .

Single camera stream start

Run the following command in the device terminal:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! 'video/x-raw,
format=NV12,\
width=1280,height=720,framerate=30/1' ! fakesink
```

The following command starts the camera with 720p at 30 FPS configuration. The frame coming from the camera sensor is thrown away by fakesink. If the gst pipeline status is changed to “PLAYING” as shown below, this indicates that the camera is running. Since this command dumps camera frames to fakesink, nothing will be saved on the device.

```
gbm_create_device(187): Info: backend name is: msm_drm
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
```

To stop the camera, press **CTRL+C**.

Video encoding

1. Run the following command in the device terminal:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/
mux_avc.mp4
```

This command starts the camera with 720p at 30 FPS configuration and saves it as a video file after h264 video encoding. If the gst pipeline status is changed to “PLAYING”, this indicates the camera is running.

To stop the camera, press **CTRL+C**.

2. /opt/mux_avc.mp4 is generated on the device. The recorded content can be pulled from the device by running the following scp command on the host PC:

```
$ scp -r root@[ip-addr]:/opt/mux_avc. mp4 .
```

Video encoding and snapshot

Note:

gst-pipeline-app is currently not enabled from GStreamer.

QCS8275 doesn't have a hardware JPEG encoder. The snapshot use case works using a software JPEG encoder algorithm that runs on the CPU.

1. Run the following command in the device terminal:

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc camera=0 ! video/x-raw,
format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 !
queue ! v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate_mode=0,
video_bitrate=6000000;" ! queue ! \
h264parse ! mp4mux ! queue ! filesink location="/opt/mux_720_avc.
mp4" camsrc.image_1 ! "image/jpeg,\
width=1280,height=720,framerate=30/1" ! multifilesink location=/
opt/frame%d.jpg async=false sync=true \
enable-last-sample=false
```

2. Press **Enter**. This command will print the following menu and wait for user input.

```
##### MENU #####
### #####

===== Pipeline
Controls=====
(0)      NULL: Set the pipeline into NULL state
(1)      READY: Set the pipeline into READY state
(2)      PAUSED: Set the pipeline into PAUSED state
(3)      PLAYING: Set the pipeline into PLAYING state
=====
Other=====
(p)      Plugin Mode: Choose a plugin which to control
(q)      Quit : Exit the application

Choose an option:
```

3. Use the following menu steps to take a snapshot while recording video.

```
(1) ready -> (3) Playing -> (p)Plugin Mode : Select (11)camerasrc
->
```

```
(38) capture-image -> (1): still - Snapshot ->(1) Snapshot count (
'guint' value for arg1)
```

4. To stop the camera, press **Enter**, press **b** (back), and then press **q** (quit). The recorded video file and snapshot images are saved in `/opt/`. The recorded content can be pulled from the device by running the following scp command on the host PC:

```
$ scp -r root@[ip-addr]:/opt/<file name> .
```

MIPI multi-camera streaming

There are three MIPI CSI slots present on the RB4 main board. Connect OV9282 cameras to MIPI CSI ports. You can run any two OV9282 cameras concurrently.

Run the following GST command in two different terminals with camera = 0, 1 to validate MIPI camera concurrency. Use different file names when saving the encoded file of each camera.

```
gst-launch-1.0 -e qtigmmfsrc name=camsrc camera=0 ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls, video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/mux_
avc.mp4
```

You can also run all three MIPI cameras concurrently using the offline IFE feature. Refer to [Support multi-camera using offline IFE](#) for information on three camera validation.

GMSL multi-camera streaming

A single GMSL port contains four slots, allowing connection of four GMSL cameras to a single CSI using different virtual channel IDs. The four cameras connected on one CSI are treated as group. A group of cameras can be run concurrently using the [per port grouping feature](#).

IQ615

GStreamer app support is currently not enabled in this release. Once enabled, the GStreamer app can be validated by performing the following steps in a device shell:

1. Create the directory `/var/cache/camera/` if it does not already exist.
2. Create `camxoverridesettings.txt` inside `/var/cache/camera/`.
3. Add the following override settings:

```
FDClient=FDSWOnly
FDProcessingControl=FDProcessingControlForceDisable
disableFDSegment=TRUE
enableDualIFE=FALSE
enableEarlyBitMLSkellLoad=FALSE
enableMCTF=FALSE
overrideForceSensorMode=2
```

3.3.3 Other GStreamer Samples

The QIM SDK includes [GStreamer sample applications for camera](#) and sample applications for [AI/ML and other multimedia applications](#).

Note:

Before using the sample applications, ensure that the installation prerequisites for `gst-launch-1.0` and GStreamer plugins are met.

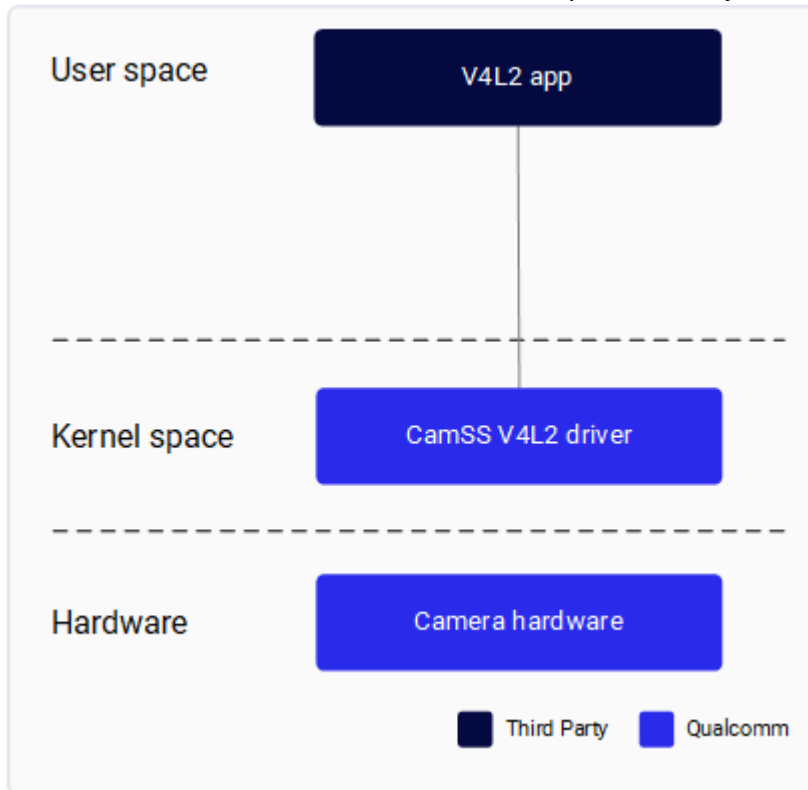
See [Multimedia use case examples](#) for examples using `gst-launch-1.0`.

3.3.4 Stream camera with the V4L2 API

To use the V4L2 API, flash the Config1 image. For overlay configuration details, see [Overlay Configurations](#).

QCS6490

The V4L2 framework within the Linux kernel supports video devices. It provides an API that allows user space applications to interact with devices such as cameras and video capture cards. More information on V4L2 is available from kernel.org. Qualcomm supports the V4L2 interface camera ISP driver for raw frame dump functionality in the upstream kernel.

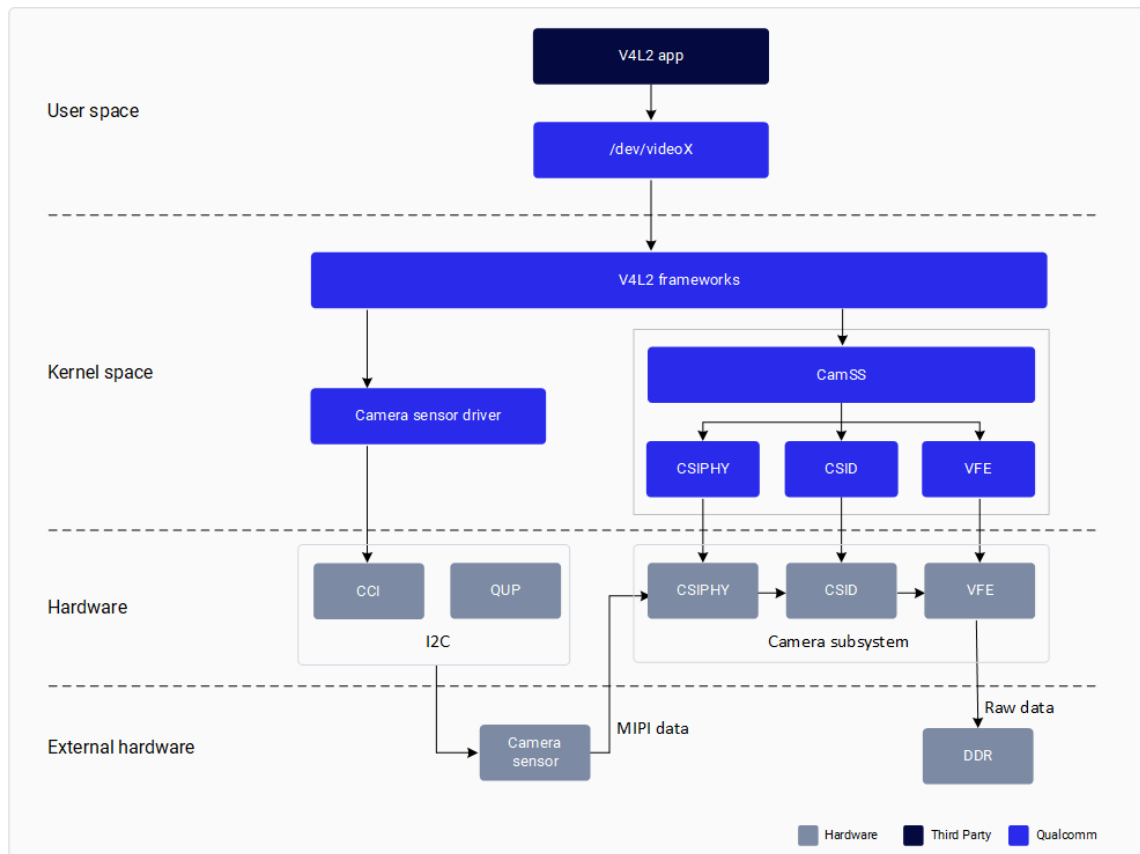
**CAMSS driver**

The Qualcomm camera subsystem (CAMSS) driver in the upstream kernel implements the V4L2, media controller, and V4L2 subdev interfaces.

Camera sensors using the V4L2 subdev interface in the kernel are supported.

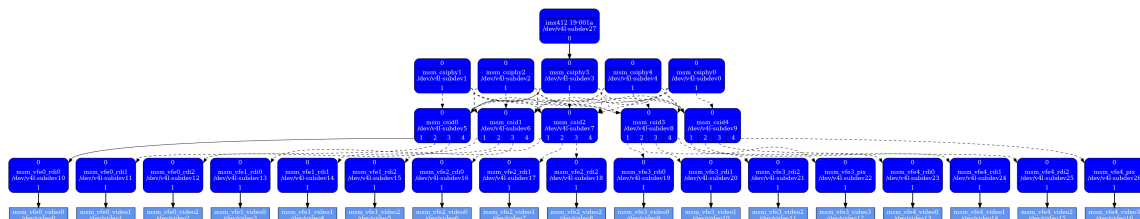
The CAMSS driver consists of:

- CSIPHY module – Handles the physical layer of the CSI2 receivers. A separate camera sensor can be connected to each CSIPHY module.
- CSI Decoder (CSID) module – Handles the protocol and application layers of the CSI2 receivers. A CSID can decode a data stream from any CSIPHY.
- Video Front End (VFE) module – Represents the Image Front End (IFE) that contains Raw Dump Interface (RDI) input interfaces that bypass the image processing pipeline. The VFE also contains the AXI bus interface which writes output data to memory.



The CAMSS driver implements the V4L2 interface. Each CSIPHY, CSID, and VFE module is represented by a single V4L2 sub-device.

As shown in the following diagram, each CSIPHY can connect to each CSID. Each CSID can connect to each VFE. Each RDI port has an individual video node.



CAMSS V4L2 drivers are in `<kernel_root>/drivers/media/platform/qcom/camss`. `<kernel_root>` is the directory of the Linux kernel.

V4L2 sample application - Yavta

This section describes how to capture raw frame data using an application that supports the V4L2 interface.

Note:

Connect the IMX577 MIPI camera sensor to the CAM3 slot of the RB3 device.

Note:

Yavta is available by default as part of the image; no explicit installation is required.

Verify the CAMSS module

The IMX412 and CAMSS modules are loaded by default. Verify that the `qcom_camss` and `IMX412` modules are loaded by running the following `lsmod` commands:

```
root@rb3gen2-core-kit:/# lsmod |grep -iE "qcom_camss|imx412"
imx412                16384  0
qcom_camss            569344  20
```

The expected result is that the `IMX412` sensor driver and `qcom_camss` camera subsystem modules are loaded successfully, confirming the upstream camera stack is active.

Check the media node number

Run the following command to print the media device node number for the `CAMSS` driver.

If `/dev/media0` does not list the `qcom-camss` driver, try it with `/dev/media1`.

```
# media-ctl -p -d /dev/media0 | grep camss
driver      qcom-camss
bus info    platform:acaf000.camss
```

Check the media device graph

Run the following command to check the media device graph:

```
media-ctl -p -d /dev/media'[x]'
```

```
entity 52: msm_vfe0_rdi0 (2 pads, 8 links, 0 routes)
  type V4L2 subdev subtype Unknown flags 0
  device node name /dev/v4l-subdev10
  pad0: SINK
    [stream:0 fmt:SRGB10_1X10/4056x3040 field:none colorspace:srgb xfer:srgb quantization:full-range]
    <- "msm_csid0":1 [ENABLED]
    <- "msm_csid1":1 []
    <- "msm_csid2":1 []
    <- "msm_csid3":1 []
    <- "msm_csid4":1 []
    <- "msm_csid5":1 []
    <- "msm_csid6":1 []
  pad1: SOURCE
    [stream:0 fmt:SRGB10_1X10/4056x3040 field:none colorspace:srgb xfer:srgb quantization:full-range]
    -> "msm_vfe0_video0":0 [ENABLED,IMMUTABLE]
```

Find the sensor name

Run the following command to print the sensor name to the terminal:

```
# cat /sys/dev/char/81\:*/*name | grep imx
imx577 21-001a
```

Configure the media controller

The media controller utility (media-ctl) is a [V4L2 utility](#) used to configure camera subsystem subdevices. Use `media-ctl --help` to print usage information.

Note:

Replace [x] with the number found via :ref. <Check the media node number>. For example,
`media-ctl -d /dev/media1 --reset`.

1. Reset all links to inactive:

```
# media-ctl -d /dev/media[x] --reset
```

2. Configure the camera sensor format and resolution on pipeline nodes:

```
# media-ctl -d /dev/media[x] -V '"imx577 17-001a":0[fmt:SRGB10/4056x3040 field:none]'
```

3. Configure CSIPHY with 4056x3040 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_csiphy3":0[fmt:SRGGB10/
4056x3040]'
# media-ctl -d /dev/media[x] -V '"msm_csiphy3":1[fmt: SRGGB10/
4056x3040]'
```

4. Configure CSID with 4056x3040 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_csid0":0[fmt:SRGGB10/4056x3040]
'
# media-ctl -d /dev/media[x] -V '"msm_csid0":1[fmt:SRGGB10/ 4056x3040]
'
```

5. Configure ISP with 3840x2160 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":0[fmt:SRGGB10/
4056x3040] '
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":1[fmt: SRGGB10/
4056x3040]'
```

6. Link the pipeline:

```
# media-ctl -d /dev/media[x] -l '"msm_csiphy3":1->"msm_csid0":0[1] '
# media-ctl -d /dev/media[x] -l '"msm_csid0":1->"msm_vfe0_rdi0":0[1]'
```

Capture images

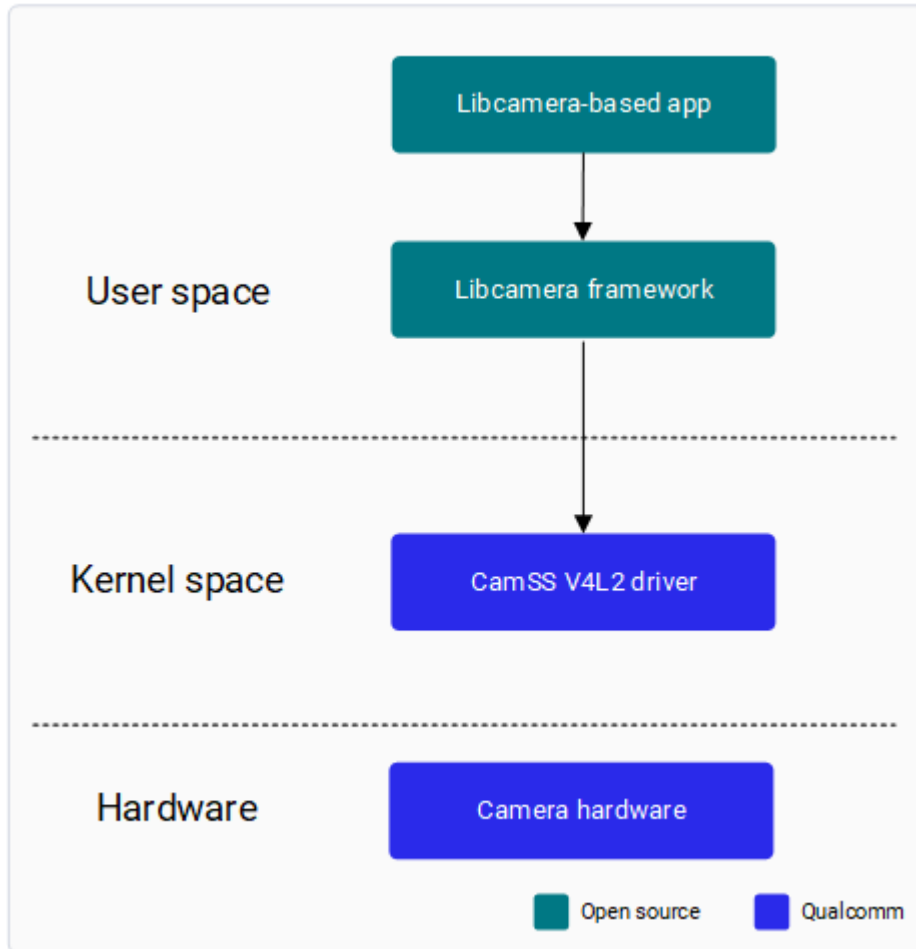
The [Yavta test application](#) validates the camera using the V4L2 interface. Run Yavta to capture images:

```
# yavta -B capture-mplane -c -I -n 5 -f SRGGB10P -s 4056x3040 -F /dev/
video0 --capture=5 --file='frame-#.raw'
```

V4L2 sample application - libcamera

libcamera is an open-source software framework. It handles control of the V4L2 camera interface and exposes a native C++ API to upper layers. The applications and upper-level frameworks run based on the [libcamera framework](#).

See [libcamera architecture](#) for more detail about the libcamera architecture.



libcamera is validated using the [cam utility](#).

Capture raw frame data

The following steps describe how to capture raw frame data with the camera utility app using the V4L2 interface.

Note:

Connect the IMX577 MIPI camera sensor to the CAM3 slot of the RB3 device.

Note:

libcamera is available by default as part of the image; no explicit installation is required.

Verify the CAMSS module

The IMX412 and CAMSS modules are loaded by default. Verify that the qcom_camss and IMX412 modules are loaded by running the following lsmod command:

```
# lsmod | grep qcom_camss
# lsmod | grep imx412
```

Delete uncalibrated.yaml

```
# rm -rf /usr/share/libcamera/ipa/simple/uncalibrated.yaml
```

Run the cam utility

```
# cam -c 1 --capture=10 --file='frame-#.raw'
```

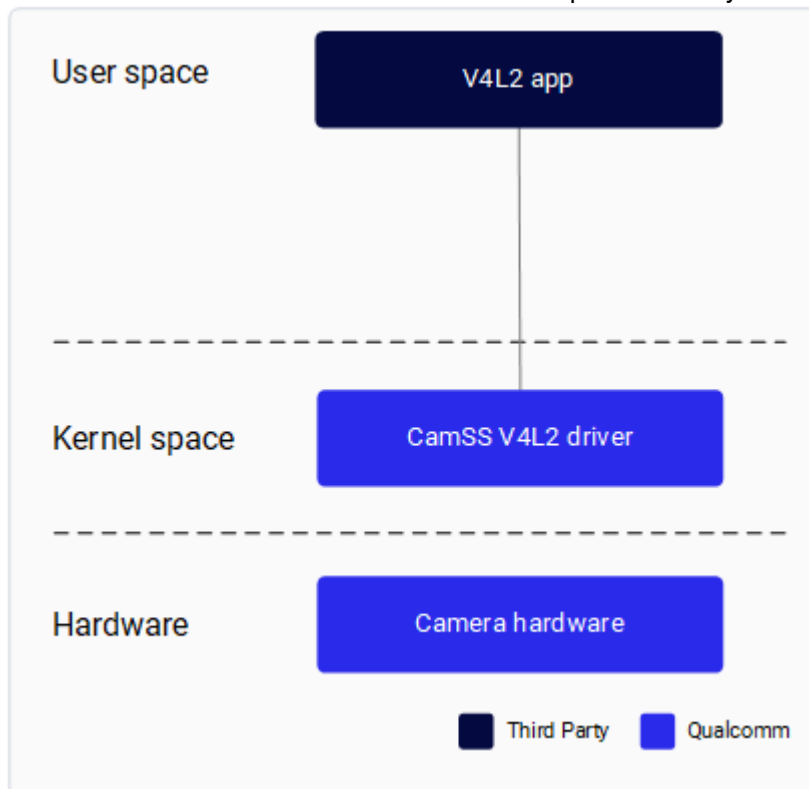
This runs the utility and captures 10 raw frames in the root directory and saves them into files.

Check the console log messages for information about the resolution and format of the dumped images.

```
Using camera /base/soc@0/cci@ac4b000/i2c-bus@1/camera@1a as cam0 [0:15:01.
067615799] [2155] INFO Camera camera.cpp:945 configuring streams:
(0) 4056x3040-SRGGB10_CSI2P
cam0: Capture 10 frames
```

QCS9075

The V4L2 framework within the Linux kernel supports video devices. It provides an API that allows user space applications to interact with devices such as cameras and video capture cards. More information on V4L2 is available from kernel.org. Qualcomm supports the V4L2 interface camera ISP driver for raw frame dump functionality in the upstream kernel.

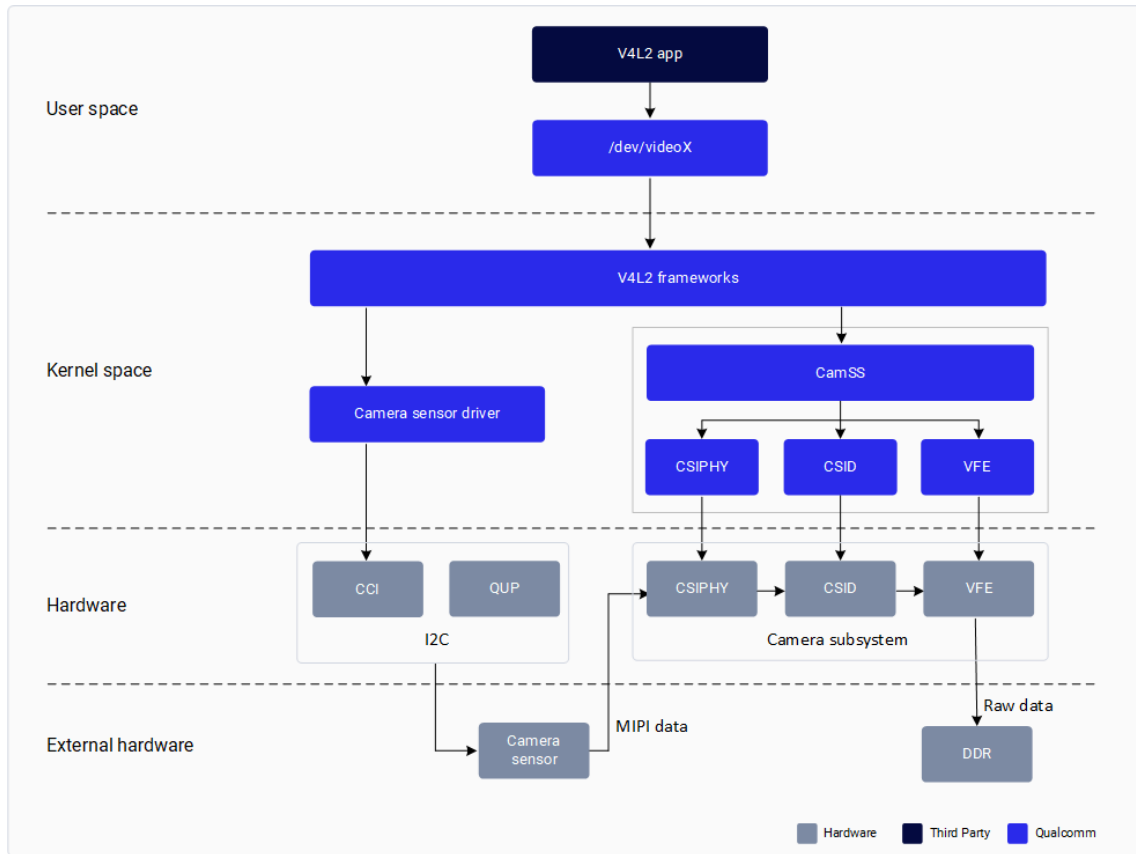
**CAMSS driver**

The Qualcomm camera subsystem (CAMSS) driver in the upstream kernel implements the V4L2, media controller, and V4L2 subdev interfaces.

Camera sensors using the V4L2 subdev interface in the kernel are supported.

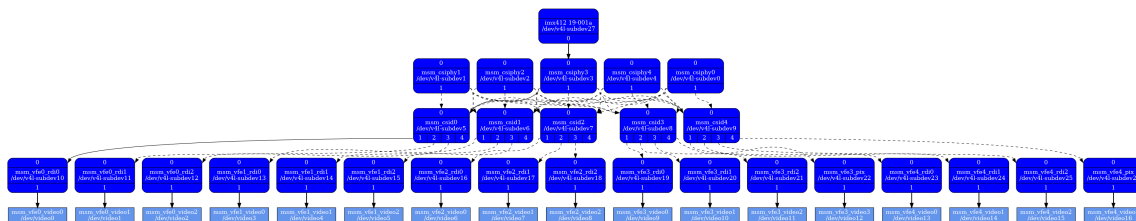
The CAMSS driver consists of:

- CSIPHY module – Handles the physical layer of the CSI2 receivers. A separate camera sensor can be connected to each CSIPHY module.
- CSI Decoder (CSID) module – Handles the protocol and application layers of the CSI2 receivers. A CSID can decode a data stream from any CSIPHY.
- Video Front End (VFE) module – Represents the Image Front End (IFE) that contains Raw Dump Interface (RDI) input interfaces that bypass the image processing pipeline. The VFE also contains the AXI bus interface which writes output data to memory.



The CAMSS driver implements the V4L2 interface. Each CSIPHY, CSID, and VFE module is represented by a single V4L2 sub-device.

As shown in the following diagram, each CSIPHY can connect to each CSID. Each CSID can connect to each VFE. Each RDI port has an individual video node.



CAMSS V4L2 drivers are in `<kernel_root>/drivers/media/platform/qcom/camss`. `<kernel_root>` is the directory of the Linux kernel.

V4L2 sample application - Yavta

This section describes how to capture raw frame data using an application that supports the V4L2 interface.

Note:

Connect the IMX577 MIPI camera sensor to the CAM1 slot of the RB8 device.

Note:

Yavta is available by default as part of the image; no explicit installation is required.

Verify the CAMSS module

The IMX412 and CAMSS modules are loaded by default. Verify the qcom_camss and IMX412 modules are loaded by running the following lsmod commands:

```
root@iq-9075-evk:/# lsmod |grep -iE "qcom_camss/imx412"
qcom_camss          569344  20
imx412              16384   0
```

The expected result is that the IMX412 sensor driver and qcom_camss camera subsystem modules are loaded successfully, confirming the upstream camera stack is active.

Check the media node number

Run the following command to print the media device node number for the CAMSS driver.

If /dev/media0 does not list the qcom-camss driver, try it with /dev/media1.

```
# media-ctl -p -d /dev/media1 | grep camss
driver      qcom-camss
bus info    platform:acaf000.camss
```

Check the media device graph

Run the following command to check the media device graph:

```
media-ctl -p -d /dev/media'x'
```

```
entity 52: msm_vfe0_rdi0 (2 pads, 8 links, 0 routes)
  type V4L2 subdev subtype Unknown flags 0
  device node name /dev/v4l-subdev10
  pad0: SINK
    [stream:0 fmt:SRGB10_1X10/4056x3040 field:none colorspace:srgb xfer:srgb quantization:full-range]
    <- "msm_csid0":1 [ENABLED]
    <- "msm_csid1":1 []
    <- "msm_csid2":1 []
    <- "msm_csid3":1 []
    <- "msm_csid4":1 []
    <- "msm_csid5":1 []
    <- "msm_csid6":1 []
  pad1: SOURCE
    [stream:0 fmt:SRGB10_1X10/4056x3040 field:none colorspace:srgb xfer:srgb quantization:full-range]
    -> "msm_vfe0_video0":0 [ENABLED,IMMUTABLE]
```

Find the sensor name

Run the following command to print the sensor name to the terminal:

```
# cat /sys/dev/char/81\:*/*name | grep imx
imx577 21-001a
```

Configure the media controller

The media controller utility (media-ctl) is a [V4L2 utility](#) used to configure camera subsystem subdevices. Use `media-ctl --help` to print usage information.

Note:

Replace [x] with the number found via :ref. <Check the media node number>. For example,
`media-ctl -d /dev/media1 --reset`.

1. Reset all links to inactive:

```
# media-ctl -d /dev/media[x] --reset
```

2. Configure the camera sensor format and resolution on pipeline nodes:

```
# media-ctl -d /dev/media[x] -V '"imx577 21-001a":0[fmt:SRGB10/4056x3040 field:none]'
```

3. Configure CSIPHY with 4056x3040 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_csiphy1":0[fmt:SRGB10/
4056x3040]'
# media-ctl -d /dev/media[x] -V '"msm_csiphy1":1[fmt:SRGB10/
4056x3040]'
```

4. Configure CSID with 4056x3040 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_csid0":0[fmt:SRGB10/4056x3040]
'
# media-ctl -d /dev/media[x] -V '"msm_csid0":1[fmt:SRGB10/ 4056x3040]
'
```

5. Configure ISP with 3840x2160 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":0[fmt:SRGB10/
4056x3040]'
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":1[fmt:SRGB10/
4056x3040]'
```

6. Link the pipeline:

```
# media-ctl -d /dev/media[x] -l '"msm_csiphy1":1->"msm_csid0":0[1]'
# media-ctl -d /dev/media[x] -l '"msm_csid0":1->"msm_vfe0_rdi0":0[1]'
```

Capture images

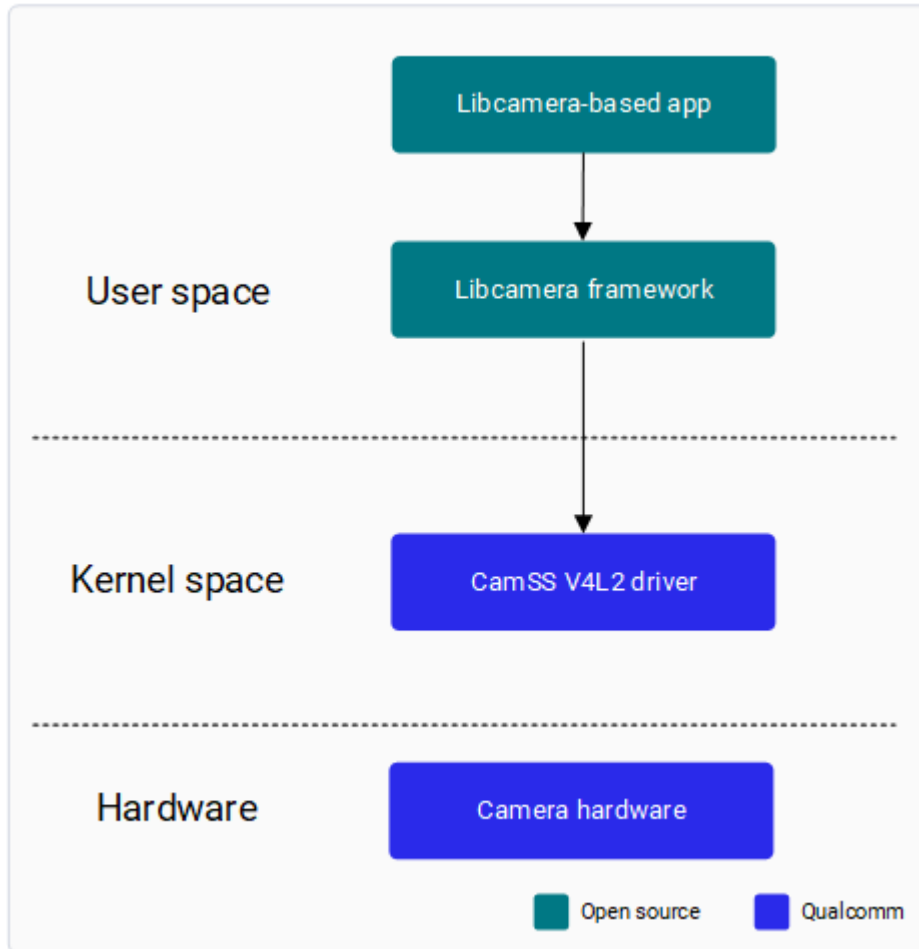
The [Yavta test application](#) validates the camera using the V4L2 interface. Run Yavta to capture images:

```
# yavta -B capture-mplane -c -I -n 5 -f SRGB10P -s 4056x3040 -F /dev/
video0 --capture=5 --file='frame-#.raw'
```

V4L2 sample application - libcamera

libcamera is an open-source software framework. It handles control of the V4L2 camera interface and exposes a native C++ API to upper layers. The applications and upper-level frameworks run based on the [libcamera framework](#).

See [libcamera architecture](#) for more detail about the libcamera architecture.



libcamera is validated using the [cam utility](#).

Capture raw frame data

The following steps describe how to capture raw frame data with the camera utility app using the V4L2 interface.

Note:

Connect the IMX577 MIPI camera sensor to the CAM1 slot of the RB8 device.

Note:

libcamera is available by default as part of the image; no explicit installation is required.

Verify the CAMSS module

The IMX412 and CAMSS modules are loaded by default. Verify the qcom_camss and IMX412 modules are loaded by running the following lsmod commands:

```
# lsmod | grep qcom_camss
# lsmod | grep imx412
```

Delete uncalibrated.yaml

```
# rm -rf /usr/share/libcamera/ipa/simple/uncalibrated.yaml
```

Run the cam utility

```
# cam -c 1 --capture=10 --file='frame-#.raw'
```

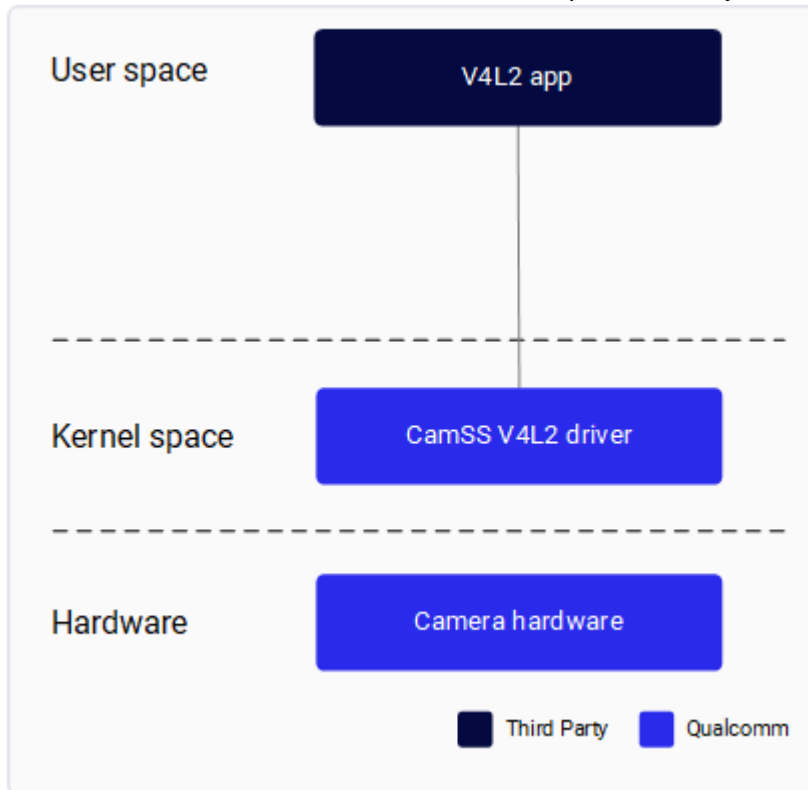
This runs the utility and captures 10 raw frames in the root directory and saves them into files.

Check the console log messages for information about the resolution and format of the dumped images.

```
Using camera /base/soc@0/cci@ac4b000/i2c-bus@1/camera@1a as cam0 [0:15:01.
067615799] [2155] INFO Camera camera.cpp:945 configuring streams:
(0) 4056x3040-SRGGB10_CSI2P
cam0: Capture 10 frames
```

QCS8275

The V4L2 framework within the Linux kernel supports video devices. It provides an API that allows user space applications to interact with devices such as cameras and video capture cards. More information on V4L2 is available from kernel.org. Qualcomm supports the V4L2 interface camera ISP driver for raw frame dump functionality in the upstream kernel.

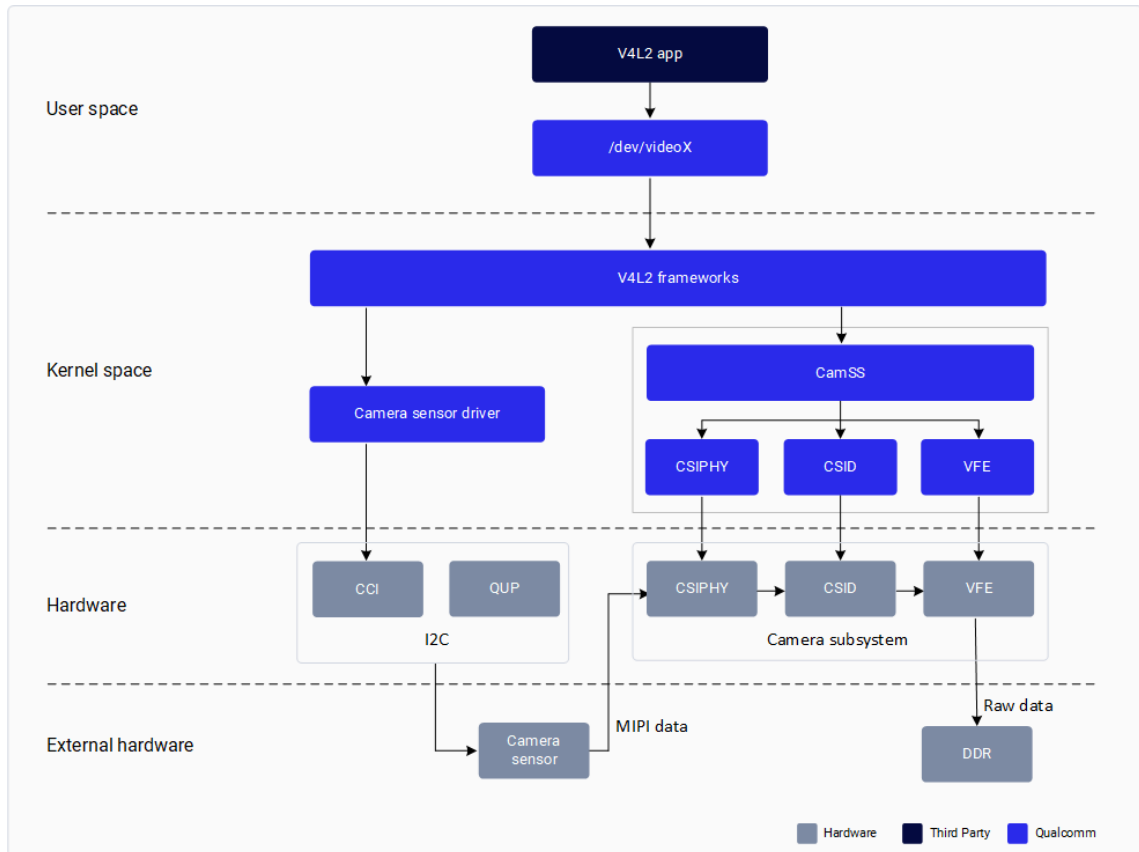
**CAMSS driver**

The Qualcomm camera subsystem (CAMSS) driver in the upstream kernel implements the V4L2, media controller, and V4L2 subdev interfaces.

Camera sensors using the V4L2 subdev interface in the kernel are supported.

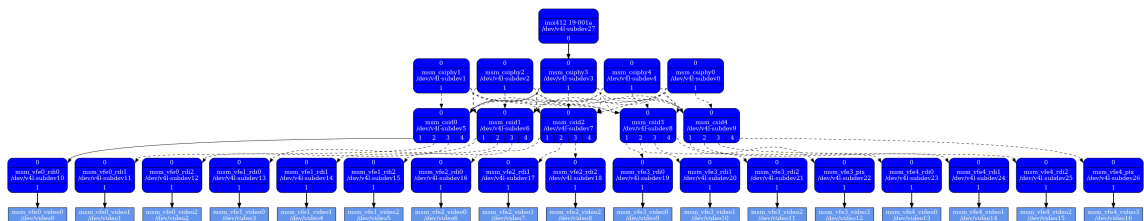
The CAMSS driver consists of:

- CSIPHY module – Handles the physical layer of the CSI2 receivers. A separate camera sensor can be connected to each CSIPHY module.
- CSI Decoder (CSID) module – Handles the protocol and application layers of the CSI2 receivers. A CSID can decode a data stream from any CSIPHY.
- Video Front End (VFE) module – Represents the Image Front End (IFE) that contains Raw Dump Interface (RDI) input interfaces that bypass the image processing pipeline. The VFE also contains the AXI bus interface which writes output data to memory.



The CAMSS driver implements the V4L2 interface. Each CSIPHY, CSID, and VFE module is represented by a single V4L2 sub-device.

As shown in the following diagram, each CSIPHY can connect to each CSID. Each CSID can connect to each VFE. Each RDI port has an individual video node.



CAMSS V4L2 drivers are in `<kernel_root>/drivers/media/platform/qcom/camss`. `<kernel_root>` is the directory of the Linux kernel.

V4L2 sample application - Yavta

This section describes how to capture raw frame data using an application that supports the V4L2 interface.

Note:

Connect the IMX577 MIPI camera sensor to the CAM1 slot of the RB4 device.

Note:

Yavta is available by default as part of the image; no explicit installation is required.

Verify the CAMSS module

The IMX412 and CAMSS modules are loaded by default. Verify the qcom_camss and IMX412 modules are loaded by running the following lsmod commands:

```
root@iq-8275-evk:/# lsmod |grep -iE "qcom_camss|imx412"
qcom_camss          569344  20
imx412              16384   0
```

The expected result is that the IMX412 sensor driver and qcom_camss camera subsystem modules are loaded successfully, confirming the upstream camera stack is active.

Check the media node number

Run the following command to print the media device node number for the CAMSS driver.

If /dev/media0 does not list the qcom-camss driver, try it with /dev/media1.

```
# media-ctl -p -d /dev/media1 | grep camss
driver          qcom-camss
bus info        platform:acaf000.camss
```

Check the media device graph

Run the following command to check the media device graph:

```
media-ctl -p -d /dev/media'[x']
```

```
entity 52: msm_vfe0_rdi0 (2 pads, 8 links, 0 routes)
  type V4L2 subdev subtype Unknown flags 0
  device node name /dev/v4l-subdev10
  pad0: SINK
    [stream:0 fmt:SRGB10_1X10/4056x3040 field:none colorspace:srgb xfer:srgb quantization:full-range]
    <- "msm_csid0":1 [ENABLED]
    <- "msm_csid1":1 []
    <- "msm_csid2":1 []
    <- "msm_csid3":1 []
    <- "msm_csid4":1 []
    <- "msm_csid5":1 []
    <- "msm_csid6":1 []
  pad1: SOURCE
    [stream:0 fmt:SRGB10_1X10/4056x3040 field:none colorspace:srgb xfer:srgb quantization:full-range]
    -> "msm_vfe0_video0":0 [ENABLED,IMMUTABLE]
```

Find the sensor name

Run the following command to print the sensor name to the terminal:

```
# cat /sys/dev/char/81\:*/*name | grep imx
imx577 21-001a
```

Configure the media controller

The media controller utility (media-ctl) is a [V4L2 utility](#) used to configure camera subsystem subdevices. Use `media-ctl --help` to print usage information.

Note:

Replace [x] with the number found via :ref. <Check the media node number>. For example,
`media-ctl -d /dev/media1 --reset`.

1. Reset all links to inactive:

```
# media-ctl -d /dev/media[x] --reset
```

2. Configure the camera sensor format and resolution on pipeline nodes:

```
# media-ctl -d /dev/media[x] -V '"imx577 21-001a":0[fmt:SRGB10/4056x3040 field:none]'
```

3. Configure CSIPHY with 4056x3040 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_csiphy1":0[fmt:SRGGB10/
4056x3040]'
# media-ctl -d /dev/media[x] -V '"msm_csiphy1":1[fmt: SRGGB10/
4056x3040]'
```

4. Configure CSID with 4056x3040 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_csid0":0[fmt:SRGGB10/4056x3040]
'
# media-ctl -d /dev/media[x] -V '"msm_csid0":1[fmt:SRGGB10/ 4056x3040]
'
```

5. Configure ISP with 3840x2160 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":0[fmt:SRGGB10/
4056x3040] '
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":1[fmt: SRGGB10/
4056x3040]'
```

6. Link the pipeline:

```
# media-ctl -d /dev/media[x] -l '"msm_csiphy1":1->"msm_csid0":0[1] '
# media-ctl -d /dev/media[x] -l '"msm_csid0":1->"msm_vfe0_rdi0":0[1]'
```

Capture images

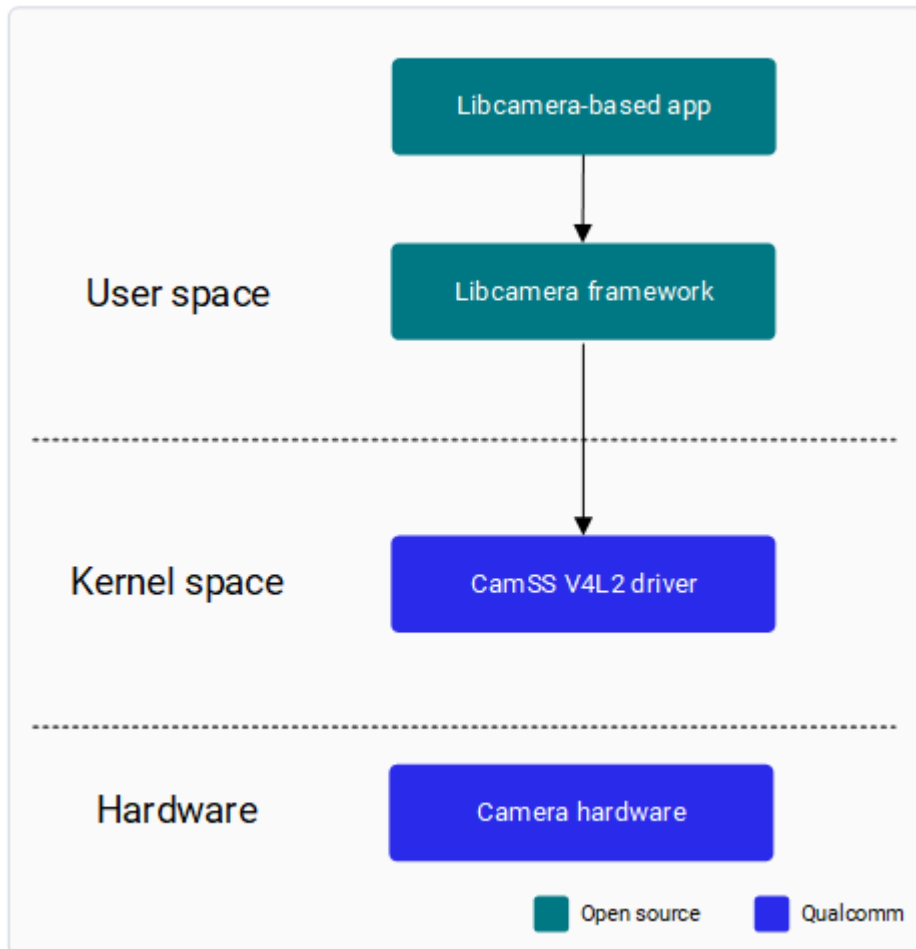
The [Yavta test application](#) validates the camera using the V4L2 interface. Run Yavta to capture images:

```
# yavta -B capture-mplane -c -I -n 5 -f SRGGB10P -s 4056x3040 -F /dev/
video0 --capture=5 --file='frame-#.raw'
```

V4L2 sample application - libcamera

libcamera is an open-source software framework. It handles control of the V4L2 camera interface and exposes a native C++ API to upper layers. The applications and upper-level frameworks run based on the [libcamera framework](#).

See [libcamera architecture](#) for more detail about the libcamera architecture.



libcamera is validated using the [cam utility](#).

Capture raw frame data

The following steps describe how to capture raw frame data with the camera utility app using the V4L2 interface.

Note:

Connect the IMX577 MIPI camera sensor to the CAM1 slot of the RB4 device.

Note:

libcamera is available by default as part of the image; no explicit installation is required.

Verify the CAMSS module

The IMX412 and CAMSS modules are loaded by default. Verify the qcom_camss and IMX412 modules are loaded by running the following lsmod commands:

```
# lsmod | grep qcom_camss
# lsmod | grep imx412
```

Delete uncalibrated.yaml

```
# rm -rf /usr/share/libcamera/ipa/simple/uncalibrated.yaml
```

Run the cam utility

```
# cam -c 1 --capture=10 --file='frame-#.raw'
```

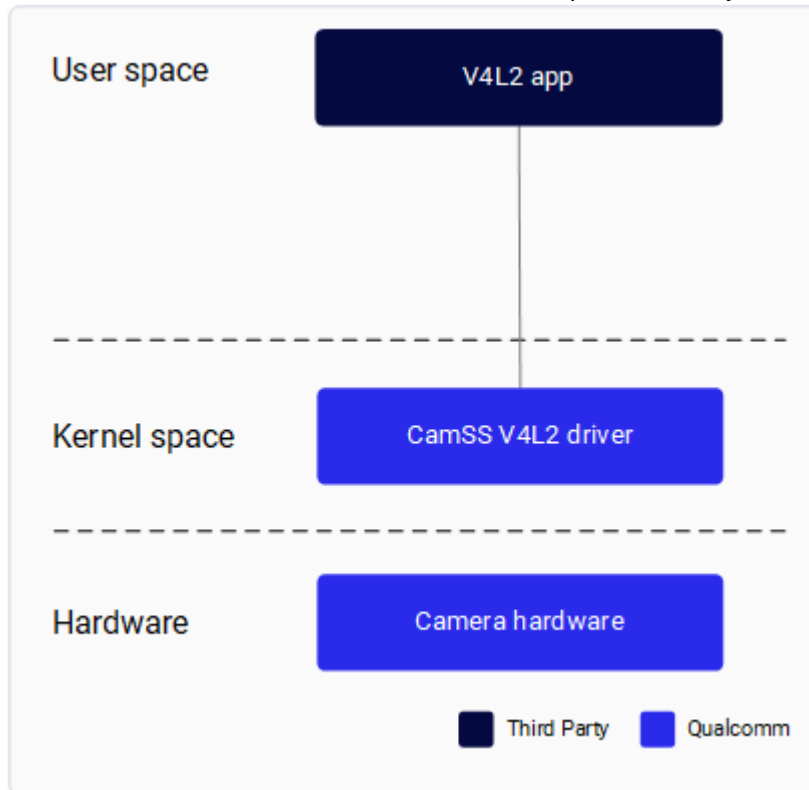
This runs the utility and captures 10 raw frames in the root directory and saves them into files.

Check the console log messages for information about the resolution and format of the dumped images.

```
Using camera /base/soc@0/cci@ac4b000/i2c-bus@1/camera@1a as cam0 [0:15:01.
067615799] [2155] INFO Camera camera.cpp:945 configuring streams:
(0) 4056x3040-SRGGB10_CSI2P
cam0: Capture 10 frames
```

IQ615

The V4L2 framework within the Linux kernel supports video devices. It provides an API that allows user space applications to interact with devices such as cameras and video capture cards. More information on V4L2 is available from kernel.org. Qualcomm supports the V4L2 interface camera ISP driver for raw frame dump functionality in the upstream kernel.

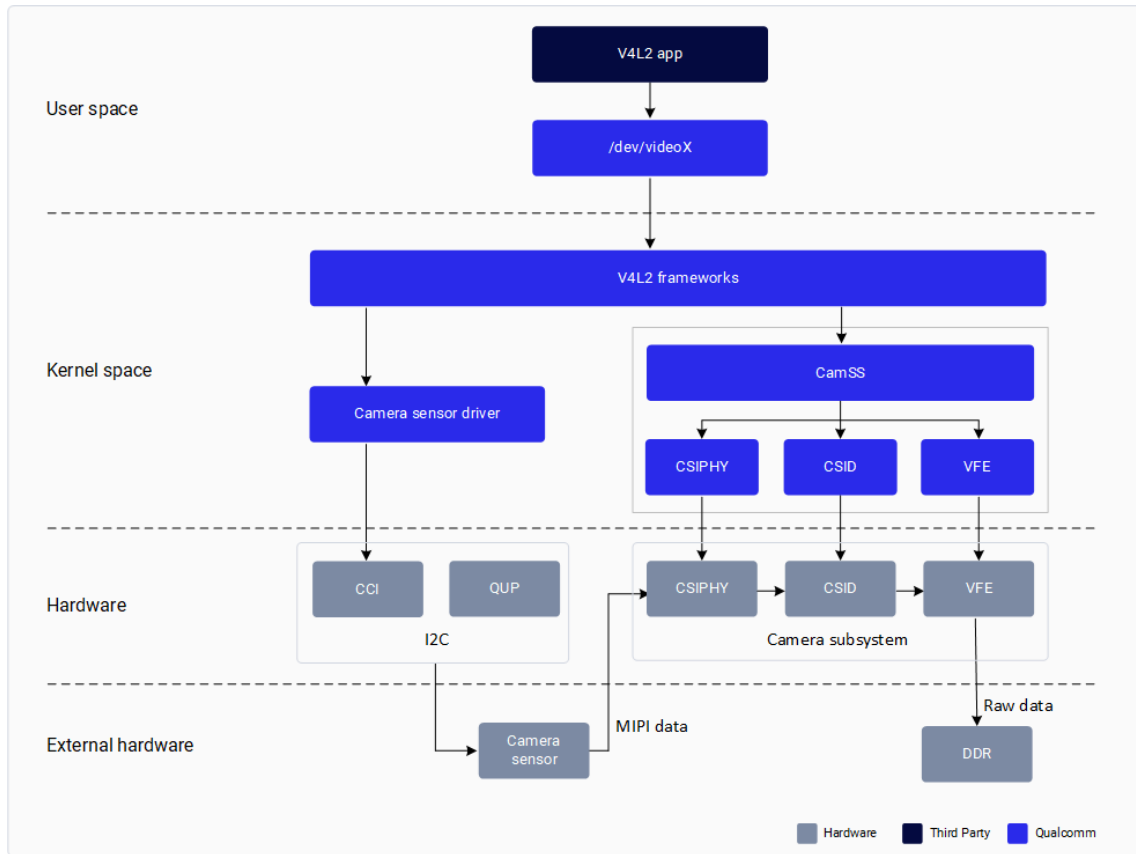
**CAMSS driver**

The Qualcomm camera subsystem (CAMSS) driver in the upstream kernel implements the V4L2, media controller, and V4L2 subdev interfaces.

Camera sensors using the V4L2 subdev interface in the kernel are supported.

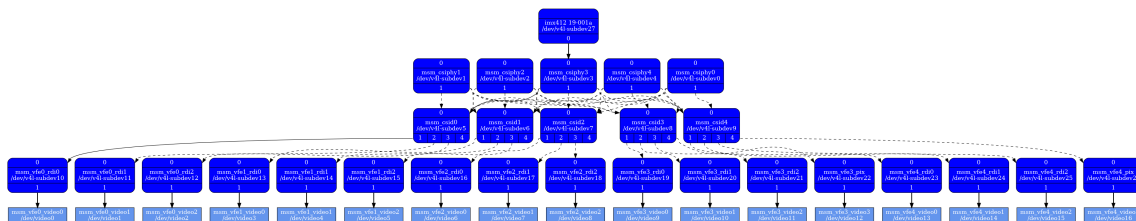
The CAMSS driver consists of:

- CSIPHY module – Handles the physical layer of the CSI2 receivers. A separate camera sensor can be connected to each CSIPHY module.
- CSI Decoder (CSID) module – Handles the protocol and application layers of the CSI2 receivers. A CSID can decode a data stream from any CSIPHY.
- Video Front End (VFE) module – Represents the Image Front End (IFE) that contains Raw Dump Interface (RDI) input interfaces that bypass the image processing pipeline. The VFE also contains the AXI bus interface which writes output data to memory.



The CAMSS driver implements the V4L2 interface. Each CSIPHY, CSID, and VFE module is represented by a single V4L2 sub-device.

As shown in the following diagram, each CSIPHY can connect to each CSID. Each CSID can connect to each VFE. Each RDI port has an individual video node.



CAMSS V4L2 drivers are in `<kernel_root>/drivers/media/platform/qcom/camss`. `<kernel_root>` is the directory of the Linux kernel.

V4L2 sample application - Yavta

This section describes how to capture raw frame data using an application that supports the V4L2 interface.

Note:

Connect the IMX577 MIPI camera sensor to the CAM1 slot of the IQ615 device.

Note:

Yavta is available by default as part of the image; no explicit installation is required.

Verify the CAMSS module

The IMX412 and CAMSS modules are loaded by default. Verify the qcom_camss and IMX412 modules are loaded by running the following lsmod commands:

```
root@iq-615-evk:/# lsmod |grep -iE "qcom_camss|imx412"
qcom_camss          569344  20
imx412              16384   0
```

The expected result is that the IMX412 sensor driver and qcom_camss camera subsystem modules are loaded successfully, confirming the upstream camera stack is active.

Check the media node number

Run the following command to print the media device node number for the CAMSS driver.

If /dev/media0 does not list the qcom-camss driver, try it with /dev/media1.

```
# media-ctl -p -d /dev/media1 | grep camss
driver          qcom-camss
bus info        platform:acaf000.camss
```

Check the media device graph

Run the following command to check the media device graph:

```
media-ctl -p -d /dev/media'[x]'
```

```
entity 52: msm_vfe0_rdi0 (2 pads, 8 links, 0 routes)
  type V4L2 subdev subtype Unknown flags 0
  device node name /dev/v4l-subdev10
  pad0: SINK
    [stream:0 fmt:SRGB10_1X10/4056x3040 field:none colorspace:srgb xfer:srgb quantization:full-range]
    <- "msm_csid0":1 [ENABLED]
    <- "msm_csid1":1 []
    <- "msm_csid2":1 []
    <- "msm_csid3":1 []
    <- "msm_csid4":1 []
    <- "msm_csid5":1 []
    <- "msm_csid6":1 []
  pad1: SOURCE
    [stream:0 fmt:SRGB10_1X10/4056x3040 field:none colorspace:srgb xfer:srgb quantization:full-range]
    -> "msm_vfe0_video0":0 [ENABLED,IMMUTABLE]
```

Find the sensor name

Run the following command to print the sensor name to the terminal:

```
# cat /sys/dev/char/81\:*/*name | grep imx
imx577 21-001a
```

Configure the media controller

The media controller utility (media-ctl) is a [V4L2 utility](#) used to configure camera subsystem subdevices. Use `media-ctl --help` to print usage information.

Note:

Replace [x] with the number found via `:ref.` <Check the media node number>. For example,

```
# media-ctl -d /dev/media1 --reset.
```

1. Reset all links to inactive:

```
# media-ctl -d /dev/media[x] --reset
```

2. Configure the camera sensor format and resolution on pipeline nodes:

```
# media-ctl -d /dev/media[x] -V '"imx577 21-001a":0[fmt:SRGB10/4056x3040 field:none]'
```

3. Configure CSIPHY with 4056x3040 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_csiphy1":0[fmt:SRGB10/
4056x3040]'
# media-ctl -d /dev/media[x] -V '"msm_csiphy1":1[fmt:SRGB10/
4056x3040]'
```

4. Configure CSID with 4056x3040 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_csid0":0[fmt:SRGB10/4056x3040]
'
# media-ctl -d /dev/media[x] -V '"msm_csid0":1[fmt:SRGB10/4056x3040]
'
```

5. Configure ISP with 3840x2160 resolution:

```
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":0[fmt:SRGB10/
4056x3040]'
# media-ctl -d /dev/media[x] -V '"msm_vfe0_rdi0":1[fmt:SRGB10/
4056x3040]'
```

6. Link the pipeline:

```
# media-ctl -d /dev/media[x] -l '"msm_csiphy1":1->"msm_csid0":0[1]'
# media-ctl -d /dev/media[x] -l '"msm_csid0":1->"msm_vfe0_rdi0":0[1]'
```

Capture images

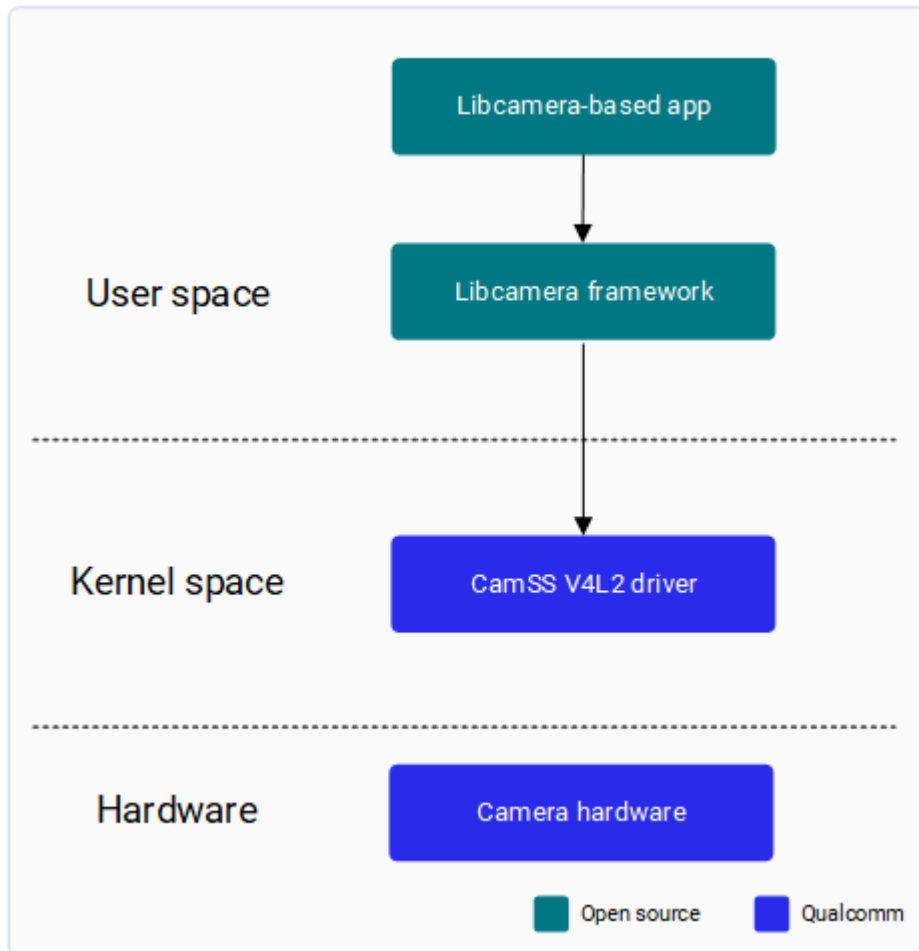
The [Yavta test application](#) validates the camera using the V4L2 interface. Run Yavta to capture images:

```
# yavta -B capture-mplane -c -I -n 5 -f SRGB10P -s 4056x3040 -F /dev/
video0 --capture=5 --file='frame-#.raw'
```

V4L2 sample application - libcamera

libcamera is an open-source software framework. It handles control of the V4L2 camera interface and exposes a native C++ API to upper layers. The applications and upper-level frameworks run based on the [libcamera framework](#).

See [libcamera architecture](#) for more detail about the libcamera architecture.



libcamera is validated using the [cam utility](#).

Capture raw frame data

The following steps describe how to capture raw frame data with the camera utility app using the V4L2 interface.

Note:

Connect the IMX577 MIPI camera sensor to the CAM1 slot of the IQ615 device.

Note:

libcamera is available by default as part of the image; no explicit installation is required.

Verify the CAMSS module

The IMX412 and CAMSS modules are loaded by default. Verify the qcom_camss and IMX412 modules are loaded by running the following lsmod commands:

```
# lsmod | grep qcom_camss
# lsmod | grep imx412
```

Delete uncalibrated.yaml

```
# rm -rf /usr/share/libcamera/ipa/simple/uncalibrated.yaml
```

Run the cam utility

```
# cam -c 1 --capture=10 --file='frame-#.raw'
```

This runs the utility and captures 10 raw frames in the root directory and saves them into files.

Check the console log messages for information about the resolution and format of the dumped images.

```
Using camera /base/soc@0/cc1@ac4b000/i2c-bus@1/camera@1a as cam0 [0:15:01.067615799] [2155] INFO Camera camera.cpp:945 configuring streams:
(0) 4056x3040-SRGGB10_CSI2P
cam0: Capture 10 frames
```

3.4 USB camera

Qualcomm Linux devices provide driver support for USB web cameras that adhere to the USB video class (UVC) standard. The UVC video driver exposes these cameras as V4L2 video devices, which can be accessed through the character device nodes such as /dev/videoX. See [USB camera configuration](#) for an USB camera usage. The sample application [gst-usb-single-camera-app](#) can be used for USB camera validation.

3.5 Network camera

A network camera can be supported with the open source GStreamer plugin. Getting a camera stream from the network depends on the network protocols supported by the GStreamer plugin. Developers can use an open source GStreamer plugin (for example, rtspsrc) to handle camera streams from the network and create a pipeline as needed. See [rtspsrc](#) for plugin details.

The sample application [gst-ai-multistream-inference](#) can also be used for network camera validation. See ‘Bring up Ethernet https://docs.qualcomm.com/doc/80-70023-26/topic/bring_up-ethernet.html’ for instructions on setting up an Ethernet network connection.

For developing a GStreamer application on the Qualcomm platform, see [Qualcomm Intelligent Multimedia SDK](#).

4 Enhance camera output

4.1 Support multi-camera using offline IFE

Note:

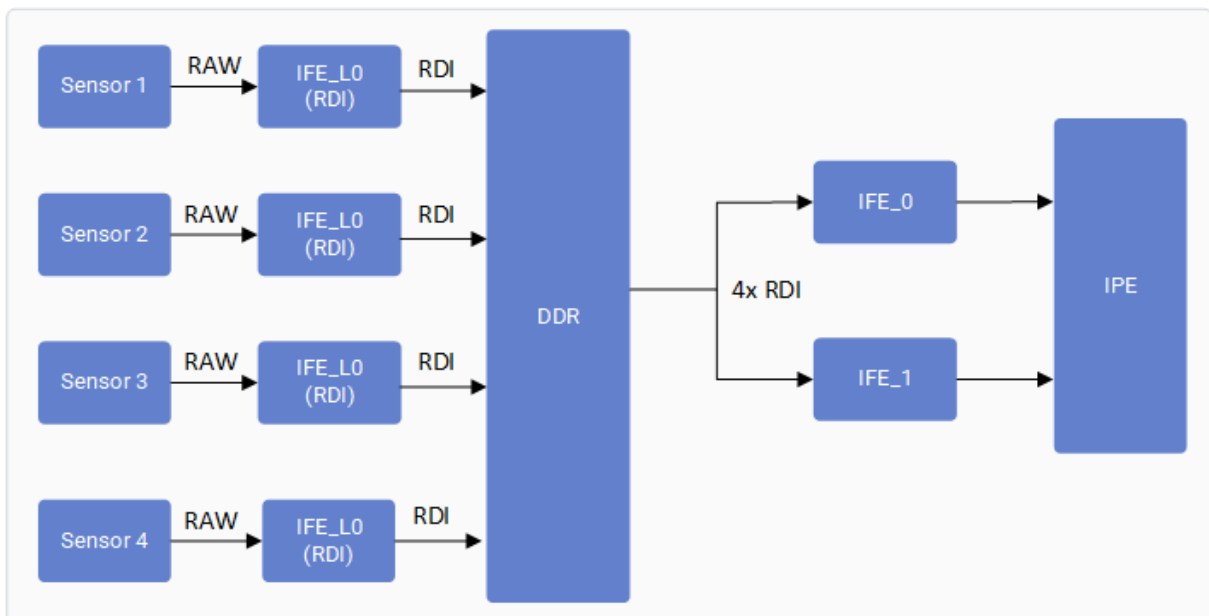
This section is applicable for QCS9075 and QCS8275.

Note:

The multi-camera using offline IFE feature is currently not enabled.

QCS9075 and QCS8275 have two IFEs that support de-bayering (bayer-to-YUV processing) of the bayer camera images in real-time. This allows for support of concurrent streaming from two bayer cameras.

The Offline IFE feature allows the IFE to run in offline mode and supports de-bayering of two cameras using single IFE. This allows for support concurrent streaming from four bayer cameras.



Each Sensor is connected to one IFE-Lite hardware instance and data is dumped to DDR using the RDI port. A single IFE hardware is being used to read data from two IFE-Lites using bus fetch engine SFE Lite and processing data in Offline mode.

With this approach, instead of directly feeding the camera frames to IFE, frames are routed from IFE_LITE and dumped in DDR. Then they're provided to IFE using the fetch engine SFE Lite for bayer-to-YUV processing.

4.1.1 Test procedure

Note:

Connect to the device console using SSH. See [How To SSH?](#) for instructions.

Ensure you have connected one of the following camera configurations:

- QCS9075: Four OV9292 MIPI cameras connected to four MIPI CSI slots (0, 1, 2, and 3)
- QCS8275: Three OV9292 MIPI cameras connected to three MIPI CSI slots (0, 1, and 2)

To collect the user log, run the following command in the device:

```
# journalctl -f > /opt/user_log.txt
```

To collect the kernel log, run the following command in the device:

```
# dmesg -w > /opt/kernel_log.txt
```

Run a GST command with `multicamera-hint=true` for each camera to enable this feature. For example:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 multicamera-hint=true ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/mux_avc.
mp4
```

Similarly run other cameras with the `multicamera-hint=true` option.

4.1.2 Log verification

Check for the following prints in user logs:

```
CamX: [CORE_CFG]3509 3556 [CORE ] camxpipeline.h:4222 SetPipelineStatus()
RealTimeFeatureZSLPreviewRawOfflineIFE_0_cam_0 status is now PipelineStatus::
STREAM_ON
```

Check for the following prints in kernel logs:

```
CAM_INFO: CAM-ISP: cam_ife_hw_mgr_print_acquire_info: 1733: 0:4:11.835 Acquired
Single IFE[0] SFE[0] OFFLINE: Y with [9 pix] [0 pd] [0 rdi] ports for ctx:1
```

4.2 Support per port grouping

Note:

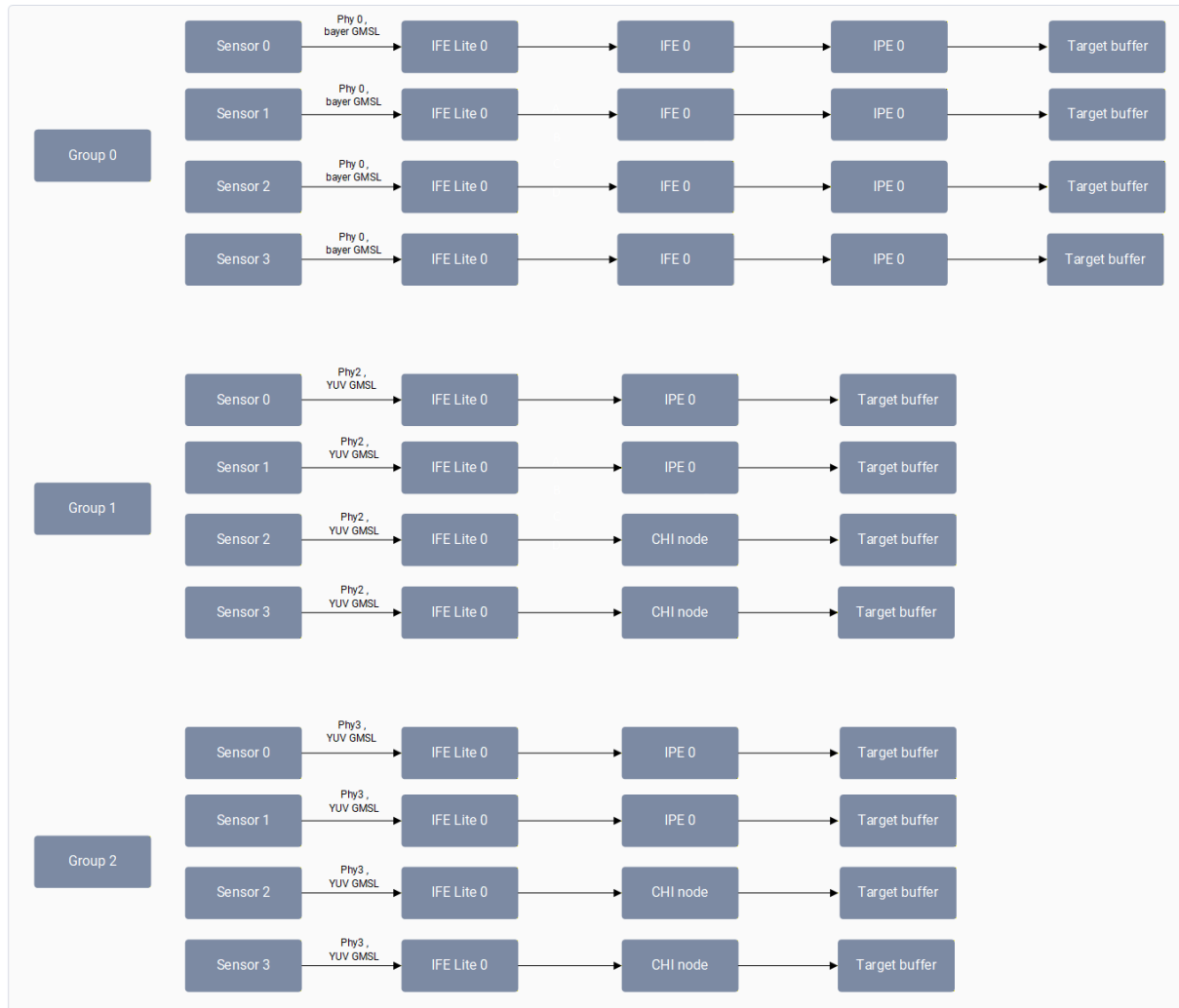
This section is applicable for QCS9075 and QCS8275.

This feature allows you to connect multiple GMSL sensors (up to 4) through one deserializer to a single CSI and IFE-Lite.

Each sensor utilizes a unique virtual (VC) channel to transmit image streams over the shared CSI interface. Each camera is handled through a separate RDI port of the IFE-Lite. With four RDI ports available in the IFE-Lite, each sensor's data is routed to an individual RDI port through VC/DT mapping.

As part of boot up, the camera software sends camera group information to the application through a vendor tag (`org.codeaurora.qcamera3.AvailableISPGroupsInfoTag`). The application then sends additional information for all the cameras it's going to open in that use case. Each sensor is mapped to a virtual RDI by having the same software pipeline per sensor (sensor -> IFE Lite). The kernel space manages different sensors connected to same CSI by mapping the virtual RDI with hardware ports of the same IFE-Lite using port sharing information. Each camera request is sent using the virtual RDI from the user space to the Kernel space. All RDI ports need to be configured before enabling streaming on any port. The application can start a camera at any time and stop at any time, as there is no restriction (cameras don't need to start together).

The following diagram provides feature design details.



In QCS9075 and QCS8275, each GMSL port/deserializer supports connection of four GMSL cameras to a single CSI. These are treated as one group.

On RB8, deserializers have the following group configuration:

- deserializer-0: 4 3F10 Bayer GMSL cameras (Group-0)
- deserializer-1: Not enabled due to hardware issue
- deserializer-2: 4 3F10 YUV GMSL cameras (Group-1)
- deserializer-3: 4 3F10 YUV GMSL cameras (Group-2)

On RB4, deserializers have the following group configuration:

- deserializer-0: 4 3F10 Bayer GMSL cameras (Group-0)
- deserializer-1: Not enabled due to hardware issue
- deserializer-2: 4 3F10 YUV GMSL cameras (Group-1)

During bootup, the camera software traverses through all the camera's sensor XML files and corresponding mode#0. As part of the static capability of each camera, the HAL provides all groups and respective camera's information through a vendor tag (`org.codeaurora.qcamera3.AvailableISPGroupsInfoTag`).

When the application first starts a camera (Open -> `Configure_streams()`), the application should give additional information for all the cameras it's going to open in the use case through the session-based parameter in the vendor tag (`org.codeaurora.qcamera3.sessionParameters -> EnabledISPGroupsConfigTag`). This includes all the groups camera information, how many cameras are going to open in each group, and, for each camera, the application needs to indicate if stats are required as well as the stream config and fps.

The KMD needs to enable all streaming paths for that camera sensor group. This needs to be called only once on the first camera's corresponding `configure_streams()`. Cameras mentioned can be a subset of the total cameras available as part of `AvailableISPGroupsInfoTag`. The application shouldn't call `configure_streams()` at same time in parallel threads as group info is passed in first configure only (that is, all other cameras `configure_streams()` can be called after the first `configure_streams()` is complete).

As long as the group configuration isn't changed, the application can start any camera mentioned in that group at any time and stop at any time, as there is no restriction (cameras don't need to start together).

If the application wants to modify the previously provided group information, then it needs to close all cameras mentioned in the previous group and send the group configuration of the use case in the first camera corresponding `configure_streams()`. If any of the previous group's cameras are still running, then the HAL will return an error.

If the application wants to disable independent port handling, then it shouldn't send any session-based param `org.codeaurora.qcamera3.sessionParameters -> EnabledISPGroupsConfigTag` in the respective `configure_streams()`. Also, the application needs to ensure the respective camera ID isn't present in the provided group information of the first `configure_streams()`. The HAL then assigns a new IFE (not combined with any of the other cameras) as per availability.

4.2.1 Validation

QCS9075

Currently on RB8, the following GMSL concurrency is supported using the `gst-camera-per-port-example` application, which runs cameras with the per port group feature.

GMSL camera concurrency	Supported resolution and FPS	Connection
Four 3F10 YUV GMSL cameras	1920x1536, 30 FPS	Connect 4 3F10 YUV GMSLs to Port-2
Four 3F10 YUV GMSL cameras	1920x1536, 30 FPS	Connect 4 3F10 YUV GMSLs to Port-3
Eight 3F10 YUV GMSL cameras	1920x1536, 30 FPS	Connect 4 3F10 YUV GMSLs to Port-2 Connect 4 3F10 YUV GMSLs to Port-3

Note:

Currently, 3F10 bayer GMSL cameras on Port-0 are not stable due to random CSI errors. Thus, it is not validated for GMSL concurrency use cases. This issue will be fixed in the next release.

Note:

The four YUV GMSL concurrency use case is supported on GMSL Port 2 and Port 3. Four bayer GMSL and eight YUV GMSL camera concurrency is not supported in this release.

To run an eight 3F10 YUV GMSL camera concurrency use case using the `gst-camera-per-port-example`:

1. Connect four 3F10 YUV GMSLs to Port-2 and four 3F10 YUV GMSLs to Port-3.
2. Run the `gst-camera-per-port-example` application.

```
# gst-camera-per-port-example --custom
```

3. Enter the camera ID's you want to open (space separated): 0 1 2 3 4 5 6 7
4. Enter the following details for all eight cameras:
 - Number of streams for camera 0: 1
 - Width for stream 1 of camera 0: 1920
 - Height for stream 1 of camera 0: 1536
 - Framerate for stream 1 of camera 0: 30
 - Number of streams for camera 1: 1
 - Width for stream 1 of camera 1: 1920
 - Height for stream 1 of camera 1: 1536
 - Framerate for stream 1 of camera 1: 30
5. Enter the GST camera pipeline command for all 8 cameras. Use different file names when saving the encoded file of each camera.

Note:

The GST command needs to be in a single line without any new line or " characters.

- Camera 0:

```
qtiqmmfsrc name=camsrc0 camera=0 video_0::type=video ! video/x-raw,format=Nv12,width=1920,height=1536,framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! queue ! v4l2h264enc capture-io-mode=4 output-io-mode=5 ! h264parse ! mp4mux ! queue ! filesink location=/opt/cam0_IOT_1536p_NV12.mp4
```

- Camera 1:

```

qtiqmmfsrc name=camsrc1 camera=1 video_0::type=video ! video/x-
raw,format=NV12,width=1920,height=1536,framerate=30/1,
interlace-mode=progressive,colorimetry=bt601 ! queue !
v4l2h264enc capture-io-mode=4 output-io-mode=5 ! h264parse !
mp4mux ! queue ! filesink location=/opt/cam1_IOT_1536p_NV12.mp4

```

The following should print in the terminal:

```

Setting pipeline gst-camera-pipeline for camera 0 to PLAYING
Pipeline is PREROLLING ...
Setting pipeline gst-camera-pipeline for camera 1 to PLAYING
Pipeline is PREROLLING ...
Pipelines are started

```

The pipeline will change from NULL to Playing state, and mp4 dumps will start for all eight GMSL cameras.

To stop camera streaming, select (q). The following logs are shown while exiting the application:

```

Quit pressed!!

gst-camera-pipeline for camera 0 Received End-of-Stream...
Setting pipeline to NULL
Pipeline state change was successful

gst-camera-pipeline for camera 1 Received End-of-Stream...
Setting pipeline to NULL
Pipeline state change was successful
g_main_loop_run ends

```

QCS8275

Currently on RB4, the following GMSL concurrency is supported using the gst-camera-per-port-example application, which runs cameras with the per port group feature.

GMSL camera concurrency	Supported resolution and FPS	Connection
Four 3F10 Bayer GMSL cameras	1824x1536, 30 FPS	Connect 4 3F10 YUV GMSLs to Port-0
Four 3F10 YUV GMSL cameras	1920x1536, 30 FPS	Connect 4 3F10 YUV GMSLs to Port-2
Four 3F10 bayer GMSL cameras + 4 YUV GMSL cameras	Bayer GMSLs:1824x1536, 30 FPS YUV GMSLs:1920x1536, 30fps	Connect 4 3F10 Bayer GMSLs to Port-0 Connect 4 3F10 YUV GMSLs to Port-2

Note:

Currently GMSL camera concurrency with the per port use case is not stable on the RB4 platform due to CSI errors. This will be fixed in next release.

Note:

Four bayer + four YUV GMSL camera concurrency is not supported in this release.

To run an eight GMSL camera concurrency use case (four 3F10 bayer GMSL cameras + four YUV GMSL cameras) using the `gst-camera-per-port-example`:

1. Connect four 3F10 bayer GMSLs to Port-0 and four 3F10 YUV GMSLs to Port-2.
2. Run the `gst-camera-per-port-example` application.

```
# gst-camera-per-port-example --custom
```

3. Enter the camera ID's you want to open (space separated): 0 1 2 3 4 5 6 7
4. Enter the following details for all eight cameras:
 - Number of streams for camera 0: 1
 - Width for stream 1 of camera 0: 1824
 - Height for stream 1 of camera 0: 1536
 - Framerate for stream 1 of camera 0: 30
 - Number of streams for camera 1: 1
 - Width for stream 1 of camera 1: 1824
 - Height for stream 1 of camera 1: 1536
 - Framerate for stream 1 of camera 1: 30
5. Enter the GST camera pipeline command for all 8 cameras. Use different file names when saving the encoded file of each camera.

Note:

The GST command needs to be in a single line without any new line or " characters.

- Camera 0:

```
qtiqmmfsrc name=camsrc0 camera=0 video_0::type=video ! video/x-raw,format=Nv12,width=1824,height=1536,framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! queue ! v4l2h264enc capture-io-mode=4 output-io-mode=5 ! h264parse ! mp4mux ! queue ! filesink location=/opt/cam0_IOT_1536p_NV12.mp4
```

- Camera 1:

```
qtiqmmfsrc name=camsrc1 camera=1 video_0::type=video ! video/x-raw,format=Nv12,width=1824,height=1536,framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! queue ! v4l2h264enc capture-io-mode=4 output-io-mode=5 ! h264parse ! mp4mux ! queue ! filesink location=/opt/cam1_IOT_1536p_NV12.mp4
```

The following should print in the terminal:

```
Setting pipeline gst-camera-pipeline for camera 0 to PLAYING
Pipeline is PREROLLING ...
Setting pipeline gst-camera-pipeline for camera 1 to PLAYING
Pipeline is PREROLLING ...
Pipelines are started
```

The pipeline will change from NULL to Playing state, and mp4 dumps will start for all eight GMSL cameras.

To stop camera streaming, select (q). The following logs are shown while exiting the application:

```
Quit pressed!!

gst-camera-pipeline for camera 0 Received End-of-Stream...
Setting pipeline to NULL
Pipeline state change was successful

gst-camera-pipeline for camera 1 Received End-of-Stream...
Setting pipeline to NULL
Pipeline state change was successful
g_main_loop_run ends
```

4.3 Support software TNR/MCTF

Note:

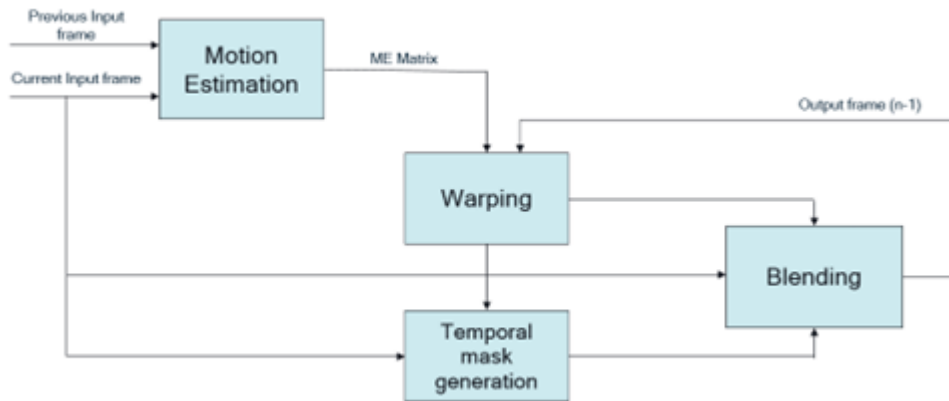
SW TNR/MCTF is not supported in this release.

Motion Compensated Temporal Filter (MCTF) is a technique used to improve preview/video quality by reducing noise and enhancing the clarity of motion.

In QCS9075 and QCS8275, the IPE hardware doesn't support MCTF. Thus, a software-based solution is implemented running on the GPU.

SWMCTF performs blending between frames in static areas to reduce noise. Additionally, the SWMCTF is the refinement of global motion compensation transform.

The following diagram shows the workflow of SWMCTF.



The first step in MCTF is to figure out the motion estimation matrix between the current frame and previous frame. The motion estimation matrix and the transform confidence measured are sent to warp and blend processing.

Next is warping. This is done on the previous output frame based on the ME matrix. An alpha mask (used for classifying background vs. local motion) is also calculated between the current and previous frame. In the end, the current input frame and the previous output are blended to generate the current output frame using alpha mask values. It also takes into consideration the noise calculated for each frame.

Use the following GST command to test software TNR:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc video_0::type=preview sw-tnr=true ! \
video/x-raw,format=NV12_Q08C,width=1280,height=720,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_prev.mp4
```

Log verification:

```
CamX: [CORE_CFG]1332 3839 [CORE ] camxpipeline.h:4222 SetPipelineStatus()
RealTimeFeatureZSLPreviewRawSWMCTF_0_cam_0 status is now PipelineStatus::
STREAM_ON
libmctf: mctf_init: 107:MCTF Version 3.14
```

4.4 Enable high dynamic range

Note:

This section is only applicable for QCS6490.

Staggered high dynamic range (SHDR) mode is a sensor feature that outputs frames with different exposure times - Long exposure frame (LEF) and short exposure frame (SEF).

- The sensor outputs a pair of two lines as one unit. The frame of LEF and the frame of SEF are output alternately in the pair of these two lines.
- The rolling shutter readout is staggered (row interleaved) so that the short integration starts immediately (within the same frame) after sampling of the long integration. This is also called Digital Overlap (DOL) mode.
- The SHDR sensors are capable of outputting LEF and SEF frames with a single virtual channel (the single frame contains both LEF and SEF in an interleaved manner) or different virtual channels (LEF and SEF frames are output separately on different virtual channels)
- On Qualcomm chipsets, there are two solutions (SHDR v3 and SHDR v2). SHDR v3 works for dual virtual channels. SHDR v2 works for single virtual channel.

Note:

Connect to the device console using SSH. See [How To SSH?](#) for instructions.

To collect the log, run the following command on the device:

```
# journalctl -f > /opt/log.txt
```

4.4.1 SHDR v3 - Dual virtual channel with two-frame exposure

Note:

SHDR v3 is currently not supported.

The SHDR v3 solution uses sensors that can send the exposure frames (LEF, SEF) on different CSI virtual channels. Long and short exposure frames are transmitted on different virtual channels.

Use the following GStreamer command to enable SHDR v3:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 vhdr=2 ! \
video/x-raw,format=NV12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_4k.mp4
```

The video file is saved at /opt/cam_4k.mp4.

To verify the SHDR v3 use case is selected, check for the following log:

```
cam-server: CamX: [CORE_CFG]891 23288 [CORE ] camxpipeline.h:3024
SetPipelineStatus() RealTimeYUVSHDR_0 status is now PipelineStatus::STREAM_ON
```

4.4.2 SHDR v2 - Single virtual channel with two-frame exposure

Note:

SHDR v2 with 4k resolution is currently not supported.

The SHDR v2 solution requires the sensor to output both LEF and SEF on a single virtual channel in an interleaved manner (also referred to as DOL mode). The rolling shutter readout is staggered (row interleaved) so that the short integration starts immediately (within the same frame) after sampling of the long integration.

Use the following GStreamer command to enable SHDR v2:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 vhdr=1 ! \
video/x-raw,format=NV12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc \
capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_4k.mp4
```

The video file is saved at `/opt/cam_4k.mp4`.

To verify the SHDR v2 use case is selected, check for the following log:

```
cam-server: CamX: [CORE_CFG]852 2070 [CORE] camxpipeline.h:3015
SetPipelineStatus()
RealTimeSHDR_0 status is now PipelineStatus::STREAM_ON
```

4.4.3 Defog feature

Note:

The defog feature is currently not enabled.

Defog is a fog detection and removal technique. This feature allows users to remove the foggy effect in poor weather conditions, such as rain, smog, haze, or fog. It provides a defogged image by improving the image quality in SHDR v2 and SHDR v3 use cases. The defog library performs defog operations with the statistics and interpolation data collected from IFE, BPS, and IPE and generates new tables for the IQ modules to apply in the next frame.

To validate the defog feature:

1. Use the following override settings in `/var/cache/camera/camxoverridesettings.txt` to verify the defog feature is working:

```
logInfoMask=0x50000
logVerboseMask=0x50000
```

2. Test the defog feature.

- Run the following command to test defog with a SHDR v2 use case:

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc vhdr=1 ! \
video/x-raw,format=Nv12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc capture-io-
mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,
video_bitrate_mode=0;" ! queue ! \
h264parse ! mp4mux ! queue ! filesink location=/opt/cam_4k.mp4
```

- Run the following command to test defog with a SHDR v3 use case:

```
gst-pipeline-app -e qtiqmmfsrc name=camsrc vhdr=2 ! \
video/x-raw,format=Nv12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc capture-io-
mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,video_
bitrate_mode=0;" ! queue ! \
h264parse ! mp4mux ! queue ! filesink location=/opt/cam_4k.mp4
```

3. Enable defog.

- Select **(1) READY**.
- Select **(3) PLAYING**.
- Press **Enter**.
- Select **(p) Plugin Mode**.
- Select **(8) camsrc**.
- Select **(24) defog-table**.
- Enter the following string to enable defog.

```
org.quic.camera.defog, enable=true, strength=3, ates_strength=1;
```

- Close the camera by selecting **(b) BACK** and then **(q) QUIT**. The video file is saved at /opt/cam_4k.mp4.

4. Check for the following logs to verify the defog feature is enabled:

- SHDR v2:

```
CamX: [ INFO]2149 2256 [CORE ] camxpipeline.cpp:5331
FillSHDRSensorModeInfo() isSHDREnabled=1 isSHDRRAWMode=1
isSHDRVCMode=0 isSHDRUseFusedStats=1 numVCSupported=0
isDefogEnablewithSHDR=1
CamX: [ VERB]2149 2182 [CHI ] camxchinodedefog.cpp:1988
SetDeFog2ConfigParams() Previous ( defog_en = 1 defog_
```

```

strength = 3 ates_strength = 1
CamX: [ VERB]2149 2182 [CHI    ] camxchinodedefog.cpp:1991
SetDeFog2ConfigParams() Current ( defog_en = 1 defog_strength
= 3 ates_strength = 1

```

- SHDR v3:

```

CamX: [ INFO]2149 2256 [CORE   ] camxpipeline.cpp:5285
FillSHDRSensorModeInfo() isSHDREnabled:1 isSHDRRAWMode:0
isSHDRVCMODE:1 isSHDRUseFusedStats:0 numVCSupported:2
isDefogEnableWithSHDR:1
CamX: [ VERB]2149 2182 [CHI    ] camxchinodedefog.cpp:1988
SetDeFog2ConfigParams() Previous ( defog_en = 1 defog_
strength = 3 ates_strength = 1
CamX: [ VERB]2149 2182 [CHI    ] camxchinodedefog.cpp:1991
SetDeFog2ConfigParams() Current ( defog_en = 1 defog_strength
= 3 ates_strength = 1

```

4.5 Enable EIS and LDC

Note:

This section is only applicable for QCS6490.

This section describes the EIS and LDC features and how to run their various use cases.

Electronic Image Stabilization (EIS) is an image enhancement technique using electronic processing. EIS minimizes blurring and compensates for device shake. EIS takes the motion data from an IMU sensor and generates the transformation matrix to compensate for device moment in all three directions.

A gyroscope sensor provides motion in pitch, yaw, and roll. The image warping library module that runs on the GPU applies warping to the image based on the generated transformation matrix. The iWarp library uses the OpenGL API for warping. Lens Distortion Correction (LDC) is the process to correct the distortion that's introduced due to fisheye lenses, which makes a straight line in a scene be captured as a curved line due to the distortion of the lens. A static mesh is generated using the calibration process of the lens.

The image warping library module that runs on the GPU applies warping to the image based on the generated static mesh. The iWarp library uses the OpenGL API for warping.

EIS and LDC can be run independently or at the same time. The following are the possible GST command options for EIS and LDC use cases:

GST command option	Description
eis=1	Enables EIS on first stream only
eis=2	Enables EIS on two streams
ldc=1	Enables LDC on all (1 or 2) streams
eis=1,ldc=1	Enables EIS and LDC on the first stream only
eis=2,ldc=1	Enables EIS and LDC on two streams

Note:

Connect to the device console using SSH. See [How To SSH?](#) for instructions.

To collect the log, run the following command on the device:

```
# journalctl -f > /opt/log.txt
```

4.5.1 EIS single stream use case

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_1080p.
mp4
```

Verify this use case is selected using the following logs:

```
cam-server[926]: [INFO]: RecorderCameraContext : OpenCamera: EIS on single
stream is ON..
cam-server[908]: CamX: [ INFO]908 981 [CHI   ] camxchinodeeisdgv26.cpp:1346
Initialize() m_nodeCaps 131072
cam-server[908]: CamX: [CORE_CFG]908 1796 [CORE       ] camxpipeline.h:3024
SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0 status is now
PipelineStatus::STREAM_ON
```

4.5.2 EIS enabled on first of two streams use case

Note:

The EIS on first of two streams use case is currently not enabled.

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 video_0::type=preview ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
```

```
v4l2h264enc capture-io-mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_prev.mp4 camsrc. ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,video_bitrate_
mode=0;" \
! h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

Verify this use case is selected using the following logs:

```
cam-server[926]: [INFO]: RecorderCameraContext : OpenCamera: EIS on single
stream is ON..
cam-server[926]: CamX: [ INFO]926 988 [CHI   ] camxchinodeeisdgv26.cpp:1346
Initialize() m_nodeCaps 131072
cam-server[926]: CamX: [CORE_CFG]926 2529 [CORE       ] camxpipeline.h:3024
SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0 status is now
PipelineStatus::STREAM_ON
```

4.5.3 EIS enabled on two streams

Note:

The EIS on two streams use case is currently not enabled.

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=2 video_0::type=preview ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_prev.mp4 camsrc. ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,video_bitrate_
mode=0;" \
! h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

Verify this use case is selected using the following logs:

```
cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera: EIS on dual stream
is ON..
cam-server[914]: CamX: [ INFO]914 960 [CHI   ] camxchinodeeisdgv26.cpp:1346
Initialize() m_nodeCaps 131072
cam-server[914]: CamX: [CORE_CFG]914 2421 [CORE       ] camxpipeline.h:3024
```

```
SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0 status is now
PipelineStatus::STREAM_ON
```

4.5.4 LDC single stream use case

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc ldc=1 ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_1080p.
mp4
```

Verify this use case is selected using the following logs:

```
cam-server[926]: [INFO]: RecorderCameraContext : OpenCamera: EIS is disabled
cam-server[926]: [INFO]: RecorderCameraContext : OpenCamera: LDC is ON..
cam-server[926]: CamX: [ INFO]926 990 [CHI   ] camxchinodeeisdgv26.cpp:1346
Initialize() m_nodeCaps 524288
cam-server[926]: CamX: [CORE_CFG]926 2132 [CORE       ] camxpipeline.h:3024
SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0 status is now
PipelineStatus::STREAM_ON
```

4.5.5 LDC enabled on two streams use case

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc ldc=1 video_0::type=preview ! \
video/x-raw,format= NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_prev.
mp4 \
camsrc. ! video/x-raw,format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc
capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,video_bitrate_
mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

Verify this use case is selected using the following logs:

```
cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera: EIS is disabled
cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera: LDC is ON..
```

```
cam-server[914]: CamX: [ INFO]914 966 [CHI    ] camxchinodeeisdgvr26.cpp:1346
Initialize() m_nodeCaps 524288
cam-server[914]: CamX: [CORE_CFG]914 1591 [CORE      ] camxpipeline.h:3024
SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0 status is now
PipelineStatus::STREAM_ON
```

4.5.6 EIS and LDC single stream use case

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 ldc=1 ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 \
! v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls="controls,
video_bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_1080p.
mp4
```

Verify this use case is selected using the following logs:

```
cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera: EIS on single
stream is ON..
cam-server[914]: [INFO]: RecorderCameraContext : OpenCamera: LDC is ON.. cam-
server[914]: CamX: [CORE_CFG]914 2621 [CORE      ] camxpipeline.h:3024
SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0 status is now
PipelineStatus::STREAM_ON
```

4.5.7 EIS and LDC enabled on first of two streams use case

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 ldc=1 video_0::type=preview ! \
video/x-raw,format= NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_prev.
mp4 camsrc. ! \
video/x-raw,format=NV12,width=1280,height=720,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,video_bitrate_
mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

Verify this use case is selected using the following logs:

```
cam-server[2661]: [INFO]: RecorderCameraContext : OpenCamera: EIS on single
stream is ON..
cam-server[2661]: CamX: [ INFO]2661 2663 [CHI          ]
camxchinodeeisdgv26.cpp:1346 Initialize() m_nodeCaps 262144
cam-server[2661]: CamX: [CORE_CFG]2661 2834 [CORE ] camxpipeline.h:3024
SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0 status is now
PipelineStatus::STREAM_ON
```

4.5.8 EIS and LDC enabled on two streams use case

Note:

The EIS and LDC enabled on two streams use case is currently not supported.

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=2 ldc=1 video_0::type=preview ! \
video/x-raw,format=NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_prev.
mp4 camsrc. ! \
video/x-raw,format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc
capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,video_bitrate_
mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

Verify this use case is selected using the following logs:

```
cam-server[882]: [INFO]: RecorderCameraContext : OpenCamera: EIS on dual stream
is ON..
cam-server[882]: CamX: [ INFO]882 977 [CHI    ] camxchinodeeisdgv26.cpp:1346
Initialize() m_nodeCaps 262144
cam-server[882]: CamX: [CORE_CFG]882 1613 [CORE      ] camxpipeline.h:3024
SetPipelineStatus() RealTimeFeatureZSLPreviewRawYuvEisIoT_0 status is now
PipelineStatus::STREAM_ON
```

4.6 Advanced feature concurrences

Note:

This section is only applicable for QCS6490.

This section describes the possible concurrency use cases for running SHDR, EIS, and LDC.

The following are the combinations available to run SHDR, EIS, and LDC in concurrency:

GST command option	Description
vhdr=2 eis=1	SHDR v3 applied on all streams. EIS applied on first stream
vhdr=2 eis=2	SHDR v3 and EIS applied on two streams
vhdr=2 ldc=1	SHDR v3+ LDC applied on all (1 or 2) streams
vhdr=1 ldc=1	SHDR v2+ LDC applied on all (1 or 2) streams
eis=1 vhdr=2 ldc=1	SHDRV3 applied on two streams. EIS+LDC applied on first stream
eis=2 vhdr=2 ldc=1	SHDRV3+EIS+LDC applied on two streams

Note:

The current release has a stability issue with SHDR v3 and EIS concurrency use cases. The GST command isn't terminating after pressing Ctrl+C. A device reboot is required to run the camera the next time.

Note:

Connect to the device console using SSH. See [How To SSH?](#) for instructions.

To collect the log, run the following command on the device:

```
# journalctl -f > /opt/log.txt
```

4.6.1 SHDR v3 and EIS concurrency use case

Note:

SHDR v3 is currently not enabled. Therefore, all concurrency use cases involving SHDRv3 are not applicable.

SHDR and EIS concurrency is enabled for SHDR v3 only. This feature uses SHDR v3 sensor mode and does SHDR v3 processing first followed by EIS processing.

Use the following GStreamer command to enable the SHDR v3+EIS single stream use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 vhdr=2 ! video/x-raw,
format=NV12,\
width=1920,height=1080,framerate=30/1 ! v4l2h264enc capture-io-mode=4 output-
io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;" !
h264parse ! \
mp4mux ! filesink location=/opt/cam_1080p.mp4
```

Verify the SHDR v3 + EIS single stream use case using the following logs:

```
cam-server[882]: [INFO]: RecorderCameraContext : OpenCamera: EIS on single
stream is ON..
cam-server[882]: CamX: [ INFO]882 975 [CHI   ] camxchinodeeisdgvt26.cpp:1346
Initialize() m_nodeCaps 131072
cam-server[882]: CamX: [CORE_CFG]882 1877 [CORE       ] camxpipeline.h:3024
SetPipelineStatus() RealTimeYUVSHDREISIoT_0 status is now PipelineStatus::
STREAM_ON
```

Use the following GStreamer command to enable the SHDR v3 + EIS on two streams use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=2 vhdr=2 video_0::type=preview ! \
video/x-raw,format= NV12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_prev.
mp4 \
camsrc. ! video/x-raw,format=NV12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc
capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,video_bitrate_
mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

Verify SHDR v3 + EIS on two streams use case using the following logs:

```
cam-server[882]: [INFO]: RecorderCameraContext : OpenCamera: EIS on dual stream
is ON..
cam-server[882]: CamX: [ INFO]882 952 [CHI ] camxchinodeeisdg26.cpp:1346
Initialize() m_nodeCaps 131072
cam-server[882]: CamX: [CORE_CFG]882 2058 [CORE ] camxpipeline.h:3024
SetPipelineStatus() RealTimeYUVSHDREISIoT_0 status is now PipelineStatus::
STREAM_ON
```

4.6.2 SHDR v3 and LDC concurrency use case

Note:

SHDR v3 is currently not enabled. Therefore, all concurrency use cases involving SHDRv3 are not applicable.

This feature uses SHDR v3 sensor mode and does SHDR v3 processing followed by LDC processing.

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc vhdr=2 ldc=1 ! video/x-raw,\
format=NV12,width=1920,height=1080,framerate=30/1 ! v4l2h264enc capture-io-
mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;" !
h264parse ! mp4mux ! \
filesink location=/opt/cam_1080p.mp4
```

Verify this use case is selected using the following logs:

```
cam-server[2089]: [INFO]: RecorderCameraContext : OpenCamera: EIS is disabled
cam-server[2089]: CamX: [ INFO]2089 2090 [CHI ]
camxchinodeeisdg26.cpp:1346 Initialize() m_nodeCaps 524288
cam-server[2089]: CamX: [CORE_CFG]2089 2324 [CORE ] camxpipeline.h:3024
SetPipelineStatus() RealTimeYUVSHDREISIoT_0 status is now PipelineStatus::
STREAM_ON
```

4.6.3 SHDR v2 and LDC concurrency use case

This feature uses SHDR v2 sensor mode and does SHDR v2 processing followed by LDC processing.

Use the following GStreamer command to enable this use case:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc vhdr=1 ldc=1 ! video/x-raw,\
format=NV12,width=1920,height=1080,framerate=30/1 ! v4l2h264enc capture-io-\
mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;" !\
h264parse ! mp4mux ! \
filesink location=/opt/cam_1080p.mp4
```

Verify this use case is selected using the following logs:

```
cam-server[2358]: [INFO]: RecorderCameraContext : OpenCamera: EIS is disabled
cam-server[2358]: CamX: [ INFO]2358 2366 [CHI ]
camxchinodeeisdgv26.cpp:1346 Initialize() m_nodeCaps 524288
cam-server[2358]: CamX: [CORE_CFG]2358 2527 [CORE ] camxpipeline.h:3024
SetPipelineStatus() RealTimeSHDR_IOTLDC_0 status is now PipelineStatus::STREAM_
ON
```

4.6.4 SHDR v3, EIS, and LDC concurrency use case

Note:

SHDR v3 is currently not enabled. Therefore, all concurrency use cases involving SHDRv3 are not applicable.

SHDR, EIS, and LDC concurrency is enabled with SHDR v3. This feature uses SHDR v3 sensor mode and does SHDR v3 processing followed by EIS and LDC processing.

Use the following GStreamer command to enable the SHDR v3, EIS, and LDC concurrency use case on a single stream:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=1 vhdr=2 ldc=1 ! video/x-raw,\
format=NV12,width=1920,height=1080,framerate=30/1 ! v4l2h264enc capture-io-\
mode=4 output-io-mode=5 \
extra-controls="controls,video_bitrate=6000000,video_bitrate_mode=0;" !\
h264parse ! mp4mux ! \
filesink location=/opt/cam_1080p.mp4
```

Verify the SHDR v3, EIS, and LDC concurrency on a single stream use case using the following logs:

```
cam-server[2555]: [INFO]: RecorderCameraContext : OpenCamera: EIS on single
stream is ON..
cam-server[2555]: CamX: [ INFO]2555 2560 [CHI      ]
camxchinodeeisdgv26.cpp:1346 Initialize() m_nodeCaps 262144
cam-server[2555]: CamX: [CORE_CFG]2555 2764 [CORE    ] camxpipeline.h:3024
SetPipelineStatus() RealTimeYUVSHDREISIoT_0 status is now PipelineStatus::
STREAM_ON
```

Use the following GStreamer command to enable the SHDR v3, EIS, and LDC concurrency use case on two streams:

```
gst-launch-1.0 -e qtiqmmfsrc name=camsrc eis=2 vhdr=2 ldc=1 video_0::
type=preview ! \
video/x-raw,format=Nv12,width=1920,height=1080,framerate=30/1 ! \
v4l2h264enc capture-io-mode=4 output-io-mode=5 extra-controls="controls,video_
bitrate=6000000,\
video_bitrate_mode=0;" ! h264parse ! mp4mux ! filesink location=/opt/cam_prev.
mp4 \
camsrc. ! video/x-raw,format=Nv12,width=1280,height=720,\
framerate=30/1,interlace-mode=progressive,colorimetry=bt601 ! v4l2h264enc
capture-io-mode=4 \
output-io-mode=5 extra-controls="controls,video_bitrate=6000000,video_bitrate_
mode=0;" ! \
h264parse ! mp4mux ! filesink location=/opt/cam_vid.mp4
```

Verify the SHDR v3, EIS, and LDC concurrency on two streams use case using the following logs:

```
cam-server[907]: [INFO]: RecorderCameraContext : OpenCamera: EIS on dual stream
is ON..
cam-server[907]: CamX: [ INFO]907 984 [CHI    ] camxchinodeeisdgv26.cpp:1346
Initialize() m_nodeCaps 262144
cam-server[907]: CamX: [CORE_CFG]907 1586 [CORE      ] camxpipeline.h:3024
SetPipelineStatus() RealTimeYUVSHDREISIoT_0 status is now PipelineStatus::
STREAM_ON
```

4.7 Enable multiple ROI streams

Note:

This section is only applicable for QCS6490.

Note:

The multiple ROI stream feature is currently not enabled.

This is an advanced camera control feature that allows the user to get multiple ROI streams from a single camera with each stream showing a different ROI from the full FOV. It supports up to three cropped ROI streams and one full ROI stream.

In case of three ROIs, the app sends five streams to configure, with one full FOV output stream, one input stream, and three ROI output streams. The real-time pipeline produces the full FOV image. The application receives the full FOV image and submits the full FOV buffer back to the camera reprocess pipeline as an input image. The reprocess pipeline generates up to three ROI images from the full FOV image.

Note:

Connect to the device console using SSH. See [How To SSH?](#) for instructions.

To collect the log, run the following command on the device:

```
# journalctl -f > /opt/log.txt
```

To verify the multiple ROI feature:

1. Connect an HDMI monitor to the device.
2. Run the following commands from an SSH terminal to set the display variables:

```
# export XDG_RUNTIME_DIR=/dev/socket/weston
# export WAYLAND_DISPLAY=wayland-1
```

3. Run the following GStreamer command:

Note:

gst-pipeline-app is currently not enabled from GStreamer.

```
gst-pipeline-app qtivcomposer name=mixer sink_0::position="<0, 0>" sink_0::
dimensions="<960, 540>" \
sink_1::position="<960, 0>" sink_1::dimensions="<960, 540>" \
sink_2::position="<0, 540>" sink_2::dimensions="<960, 540>" \
sink_3::position="<960, 540>" sink_3::dimensions="<960, 540>" mixer. ! queue
! \
waylandsink qos=false sync=false fullscreen=true \
qtiqmmfsrc input-roi-enable=true video_0::type=preview video_1::type=preview
\
video_2::type=preview video_3::type=preview video_0::reprocess-enable=true \
name=camsrc ! video/x-row,format=NV12,width=1920,height=1080,framerate=30/1 !
queue ! mixer. \
camsrc. ! video/x-row,format=NV12,width=1920,height=1080,framerate=30/1 !
queue ! mixer. \
camsrc. ! video/x-row,format=NV12,width=1920,height=1080,framerate=30/1 !
queue ! mixer. \
camsrc. ! video/x-row,format=NV12,width=1920,height=1080,framerate=30/1 !
queue ! mixer.
```

4. Select **(1) READY**.
5. Select **(3) PLAYING**. Press **Enter**.
6. Select **(p) Plugin Mode**.
7. Select **(12) camsrc**. You should see four preview streams with Full FOV.
8. Select **(32) input-roi-info**.
9. Enter the following input:

```
<1920, 0, 1920, 1080, 0, 1080, 1920, 1080, 0, 0, 1920, 1080>
```

You should see one full FOV and three ROI streams on HDMI.

To close the camera, select **(b) BACK** and then **(q) QUIT**.

Check the following UMD logs to verify multiple ROI pipeline is selected:

```
cam-server[2218]: CamX: [CORE_CFG]2218 2368 [CORE ] camxpipeline.h:3024
SetPipelineStatus() ZSLReprocessYuvToYuvMultiRoIStreams_0 status is now
PipelineStatus::STREAM_ON
```

Note:

The total bandwidth for the Full FOV stream + ROI streams needs to be less than 4K, which is the max capability of the QCS6490 chipset.

4.8 Switch linear vs. SHDR mode automatically

Note:

This section is only applicable for QCS6490.

This feature allows on-the-fly switching between Linear and SHDR v2 mode pipelines without stopping and starting the camera session.

This feature is helpful to prevent interrupting a video session when there is a SHDR vs. Linear mode switch needed based on lux value. It also helps reduce the pipeline switch latency as the pipelines (Linear and SHDR) are pre-initialized during session creation time. During mode switch, hardware resources of the pipeline are released (sensor, IFE, IPE) and then acquired and reconfigured for the new pipeline.

Note:

This feature works only for SHDR v2 and Linear mode pipelines. It will be enabled for SHDR v3 and Linear mode in a future release.

Note:

Connect to the device console using SSH. See [How To SSH?](#) for instructions.

To collect the log, run the following command on the device:

```
# journalctl -f > /opt/log.txt
```

Use `vhdr=3` in a GST command to enable this feature. To enable the feature and toggle between Linear and SHDR modes:

1. Run the `gst-camera-metadata-example` application, which can support setting the vendor tag to switch the use case during runtime. For example:

```
gst-camera-metadata-example -p "qtiqmmfsrc name=camsrc camera=0 vhdr=3 ! \
video/x-raw,format=Nv12,width=3840,height=2160,framerate=30/1,\
interlace-mode=progressive,colorimetry=bt601 ! queue ! v4l2h264enc capture-
io-mode=4 \
output-io-mode=5 ! queue ! h264parse ! mp4mux ! queue ! filesink location=/
opt/mux4k.mp4"
```

2. In the application, select **(1) READY**.
3. Select **(3) PLAYING**. The camera starts in linear mode.
4. Select **(4) META**. The following menu is shown:

```
-----MENU-----
(1)      video-metadata
(2)      image-metadata
```

```
(3)      static-metadata
(4)      session-metadata
```

5. Select **(1) video-metadata**. The following menu is shown:

```
-----video-metadata-----
(1)      List all available tags
(2)      Dump all tags values in a file
(3)      Dump custom tags values in a file
(4)      Get a tag
(5)      Set a tag
```

6. Select **(5) Set a tag**.

7. A prompt appears to enter the section name and tag name separated by spaces without quotes. Enter `org.quic.camera.videoHDRmode modeType`

8. A prompt appears to set the value. Enter 1 to switch to SHDR mode.
To toggle between Linear and SHDR modes, repeat this process and enter 1 (SHDR mode) and 0 (Linear mode) as needed.

To close the camera, select **(q) QUIT** or press **CTRL + C**.

Verify this feature using the following logs:

- Linear mode:

```
cam-server[1693]: CamX: [REQMAP]1693 2652 [CORE  ] camxsession.cpp:4811
ProcessRequest() chiFrameNum: 0 <==> requestId: 1 <==> sequenceId: 0
<==> CSLSyncId: 1 -- RealTimeFeatureZSLPreviewRawYUV_0
```

- SHDR mode:

```
cam-server[1693]: CamX: [REQMAP]1693 2652 [CORE  ] camxsession.cpp:4811
ProcessRequest() chiFrameNum: 296 <==> requestId: 297 <==> sequenceId:
296 <==> CSLSyncId: 297 -- RealTimeSHDR_0
```

5 Troubleshoot camera

Collect camera logs using one of the following commands:

```
# cat /var/log/messages > log.txt
```

Or:

```
# journalctl -f > log.txt
```

This command collects both KMD and UMD logs. To collect only KMD logs, use the following command:

```
# dmesg -w > dmesg.txt
```

5.1 Capture user mode logs

A variety of log levels available in the camera software. Based on need, developers can enable these log levels to debug issues efficiently.

Log levels are controlled through the following override settings.

Description	Override setting	Default value
Verbose log mask	logVerboseMask	0
Entry/exit log mask	logEntryExitMask	0
Info log mask	logInfoMask	0xFFFFFFFF
Warning log mask	logWarningMask	0xFFFFFFFF
System log enable	systemLogEnable	TRUE

To enable logging:

1. On first use, create a text filename `camxoverridesettings.txt`.
2. Add the required log mask to this text file. For example, `LogVerboseMask=0XFF11` and `LogInfoMask=0x1C09811`.
3. Push the modified file to `/var/cache/camera/camxoverridesettings.txt`. A device reboot may be required after changing override settings.

Example 1

Controlled debug log messages determine which groups of the camera system output logs for each log level. To enable verbose and entry or exit log settings, add the following to the `/var/cache/camera/camxoverridesettings.txt` file:

```
logVerboseMask=0xFFFFFFFF
logEntryExitMask=0xFFFFFFFF
```

Example 2

To enable verbose logs for HAL CSL groups only, enable warnings from sensor, HAL, and CSL and disable system logging:

```
logVerboseMask=0x00000480
logWarningMask=0x482
```

5.1.1 Camera UMD logs

The camera driver is implemented with debug log messages. The log messages can be filtered to limit what's displayed according to user interest (verbosity level and group). Each log comes from a specified group and has a verbosity level ranging from verbose to error. All camera debug logs follow the same format:

```
CamX: [<Verbosity Level>][<Group>] <File>:<Line Number> <Function Name>
<Message>
```

For example:

```
CamX: [INFO][CORE] camxexamplefile:123 ExampleFunction()This is the message.
```

The following verbosity levels are supported by the driver to gain insight into problems when debugging camera applications.

Error	Fatal and non-fatal errors. Errors can't guarantee successful continued operation.
Warning	Non-fatal conditions which are unexpected, but not necessarily an error. Used to indicate issues such as slow performance paths, fallback code, etc.
Entry or exit	Entry or exit API tracing. Allows simple, high-level call flow logging.
Info	Informational logs, short one-time configuration, etc.
Verbose	All other logs. Doesn't necessarily provide any useful information for debugging purposes.

5.1.2 Camera user mode logging groups

The driver is split into various logical components known as groups. Using a specific group allows you to reduce the amount of log messages sent to the component(s) you are interested in obtaining information from. This is accomplished with a bitmask to selectively choose which logs to enable from different components of the camera system.

The following table lists the UMD logging groups.

Driver group name	Value	Description
CamxLogGroupAFD	(1<< 0)	AFD
CamxLogGroupSensor	(1<< 1)	Sensor
CamxLogGroupTracker	(1<< 2)	Tracker
CamxLogGroupISP	(1<< 3)	ISP
CamxLogGroupPPProc	(1<< 4)	Post processor
CamxLogGroupMemMgr	(1 << 5)	MemMgr
CamxLogGroupPower	(1 << 6)	Power (clock, bandwidth related)
CamxLogGroupHAL	(1<< 7)	HAL
CamxLogGroupJPEG	(1<< 8)	JPEG
CamxLogGroupStats	(1<< 9)	3A algorithms
CamxLogGroupCSL	(1<< 10)	Camera service layer
CamxLogGroupApp	(1<< 11)	Application
CamxLogGroupUtils	(1<< 12)	Utilities
CamxLogGroupSync	(1<< 13)	Synchronization/mutex/fences
CamxLogGroupMemSpy	(1<< 14)	MemSpy
CamxLogGroupFormat	(1<< 15)	Format
CamxLogGroupCore	(1<< 16)	Core camera system
CamxLogGroupHWL	(1<< 17)	Hardware layer
CamxLogGroupChi	(1<< 18)	CameraHALInterface
CamxLogGroupDRQ	(1<< 19)	Deferred request queue
CamxLogGroupFD	(1<< 20)	Face detection
CamxLogGroupIQMod	(1<< 21)	IQ module
CamxLogGroupLRME	(1<< 22)	LRME
CamxLogGroupCVP	(1<< 22)	CVP
CamxLogGroupNCS	(1<< 23)	NCS
CamxLogGroupMeta	(1<< 24)	Metadata
CamxLogGroupAEC	(1<< 25)	AEC
CamxLogGroupAWB	(1<< 26)	AWB
CamxLogGroupAF	(1<< 27)	AF
CamxLogGroupSWP	(1<< 28)	SW Process
CamxLogGroupHist	(1<< 29)	HistogramProcess
CamxLogGroupBPS	(1<< 30)	BPS
CamxLogGroupDebugData	(1<< 31)	Debug data framework, 3A debug data, and tuning metadata

5.2 Capture kernel mode logs

KMD log messages are enabled by default and appear in the kernel logs. Camera KMD logs all start with the CAM prefix. Additional debugging logs are available in the KMD and are controlled through groups.

The following table lists the KMD logging groups.

KMD group	Value	Description
CAM_CDM	(1<< 0)	Camera data mover
CAM_CORE	(1<< 1)	Camera core
CAM_CPAS	(1<< 2)	Camera peripherals and support
CAM_ISP	(1<< 3)	Image signal processor
CAM_CRM	(1<< 4)	Camera request manager
CAM_SENSOR	(1<< 5)	Sensor
CAM_SMMU	(1<< 6)	Shared memory management unit
CAM_SYNC	(1<< 7)	Synchronization
CAM_ICP	(1<< 8)	Image control processor
CAM_JPEG	(1<< 9)	JPEG
CAM_FD	(1<< 10)	Face detection
CAM_LRME	(1<< 11)	Low resolution motion estimation
CAM_FLASH	(1<< 12)	Flash
CAM_ACTUATOR	(1<< 13)	Actuator
CAM_CCI	(1<< 14)	Camera control interface
CAM_CSIPHY	(1<< 15)	Camera serial interface
CAM_EEPROM	(1<< 16)	Electronically erasable programmable read-only memory
CAM_UTIL	(1<< 17)	Utilities
CAM_HFI	(1<< 18)	Host-firmware interface
CAM_CTXT	(1<< 19)	Camera context
CAMX_OIS	(1<< 20)	Optical image stabilization
CAM_RES	(1<< 21)	Camera shared resource API
CAM_MEM	(1<< 22)	For camera memory manager
CAM_IRQ_CTRL	(1<< 23)	For events in IRQ controller
CAM_REQ	(1<< 24)	Tracks a request submitted to KMD
CAM_PERF	(1<< 25)	Used for performance (clock, bandwidth, etc.) logs
CAM_CUSTOM	(1 << 26)	For custom hardware node
CAM_PRECIL	(1<<27)	Presil changes
CAM_OPE	(1<<28)	Offline processing engine
CAM_IO_ACCESS	(1<<29)	Camera I/O access(reg read/write, etc.)
CAM_SFE	(1<<30)	Streaming front-end
CAM_IR_LED	(1<<31)	IR LED

5.2.1 Enable camera kernel mode driver logs

1. To enable CAM_SENSOR and CAM_ICP debug logs in the KMD, run the following commands in an SSH shell:

- QCS6490

```
# echo 0x120 > /sys/module/camera_qcm6490/parameters/debug_md1
# cat /var/log/kern.log > name_of_kmd_logs.txt
```

- QCS9075/QCS8275

```
# echo 0x120 > /sys/module/camera_qcs9100/parameters/debug_md1
# cat /var/log/kern.log > name_of_kmd_logs.txt
```

- QCS615

```
# echo 0x120 > /sys/module/camera_qcs615/parameters/debug_md1
# cat /var/log/kern.log > name_of_kmd_logs.txt
```

2. To enable CSID IRQ logs dynamically, use the appropriate mask value(s) with the following commands.

- QCS6490

```
# mount -t debugfs none /sys/kernel/debug/
# echo MASKVALUE > /sys/kernel/debug/camera_ife/ife_csid_debug
```

- QCS9075/QCS8275/QCS615

```
# mount -t debugfs none /sys/kernel/debug/
# echo MASKVALUE > /sys/kernel/debug/camera/ife/ife_csid_debug
```

The following table lists the CSID debug groups.

CSID debug group	Value	Description
CSID_DEBUG_ENABLE_SOF_IRQ	(1<< 0)	Start of frame
CSID_DEBUG_ENABLE_EOF_IRQ	(1<< 1)	End of frame
CSID_DEBUG_ENABLE_SOT_IRQ	(1<< 2)	Start of transmission
CSID_DEBUG_ENABLE_EOT_IRQ	(1<< 3)	End of transmission
CSID_DEBUG_ENABLE_SHORT_PKT_CAPTURE	(1<< 4)	Short packet capture
CSID_DEBUG_ENABLE_LONG_PKT_CAPTURE	(1<< 5)	Long packet capture
CSID_DEBUG_ENABLE_CPHY_PKT_CAPTURE	(1<< 6)	CPHY packet capture

For example:

- To enable SOF, EOF, SOT, and EOT IRQs:

```
# echo 0xf > /sys/kernel/debug/camera/ife/ife_csid_debug
```

- To set only short packet and long packet capture IRQs:

```
# echo 0x30 > /sys/kernel/debug/camera/ife/ife_csid_debug
```

Information is logged to kmsg and can be obtained using the following command:

```
# cat /var/log/kern.log > name_of_kmd_logs.txt
```

The captured logs can help provide details about the SOF and EOF and indicate which sensor is streaming. The following is an example log:

```
CAM_INFO: CAM-ISP: cam_ife_csid_irq: 5288 CSID:1 IPP SOF received
CAM_INFO: CAM-ISP: cam_ife_csid_irq: 5300 CSID:1 IPP EOF received
CAM_INFO: CAM-ISP: cam_ife_csid_irq: 5288 CSID:1 IPP SOF received
CAM_INFO: CAM-ISP: cam_ife_csid_irq: 5300 CSID:1 IPP EOF received
CAM_INFO: CAM-ISP: cam_ife_csid_irq: 5288 CSID:1 IPP SOF received
CAM_INFO: CAM-ISP: cam_ife_csid_irq: 5300 CSID:1 IPP EOF received
```

5.2.2 Enable CSIPHY log dumps

To enable CSIPHY log dumps:

1. Use the following command to get log dumps from CSIPHY:

```
# echo 1 > /sys/module/camera_<chipset_name>/parameters/csiphy_dump
```

The information is logged to kmsg.

2. Use the following command to obtain the log dump file:

```
# cat /proc/kmsg > name_of_kmd_logs.txt
```

Note:

In these logs, <chipset name> corresponds to the following:

- qcm6490 - RB3gen2
- qcs9100 - RB8/RB4
- qcs615 - IQ-615-EVK

5.2.3 Enable CSID SOF/EOF IRQ logs

To enable CSID SOF/EOF IRQ logs, run the following commands for trace logs:

```
# mount -t debugfs none /sys/kernel/debug/
# echo 0x8 > /sys/module/camera_<chipset_name>/parameters/debug_md1
# echo 3 > /sys/kernel/debug/camera/ife/ife_csid_debug
```

Note:

In these logs, <chipset name> corresponds to the following:

- qcm6490 - RB3gen2
- qcs9100 - RB8/RB4
- qcs615 - IQ-615-EVK

5.2.4 Collect CPAS logs

To collect clock voting information in the camera log:

1. Run the following command using `camxoverridesettings.txt` for core, HAL, IFE, and power:

```
# echo "logInfoMask=0x500C8" >> /var/cache/camera/
camxoverridesettings.txt
```

2. After rebooting, enable the KMD CAM_CPAS|CAM_PERF|CAM_REQ mask:

```
# echo 0x3000004 > /sys/module/camera_<chipset_name>/parameters/debug_
mdl
# echo 0x1 > /sys/module/camera_<chipset_name>/parameters/cpas_dump
```

3. Collect the log, and run the use case:

```
# cat /var/log/messages > log.txt
# cat /proc/kmsg > kmesg.txt
```

Note:

In these logs, `<chipset name>` corresponds to the following:

- qcm6490 - RB3gen2
- qcs9100 - RB8/RB4
- qcs615 - IQ-615-EVK

5.2.5 Enable kernel run-time log mask

Run the following command to enable CRM logs:

```
# echo 0x10 > /sys/module/camera_<chipset_name>/parameters/debug_mdl
```

For example, to enable CRM, ISP, and CPAS logs:

```
# echo 0x1C > /sys/module/camera_<chipset_name>/parameters/debug_mdl
```

Note:

In these logs, `<chipset name>` corresponds to the following:

- qcm6490 - RB3gen2
- qcs9100 - RB8/RB4

- qcs615 - IQ-615-EVK

No device reboot is required. If you reboot, run this command again before opening the camera application.

The following list provides the debug bitmask for each driver.

```
CAM_CDM (1 << 0)
CAM_CORE (1 << 1)
CAM_CPAS (1 << 2)
CAM_ISP (1 << 3)
CAM_CRM (1 << 4)
CAM_SENSOR (1 << 5)
CAM_SMMU (1 << 6)
CAM_SYNC (1 << 7)
CAM_ICP (1 << 8)
CAM_JPEG (1 << 9)
CAM_FD (1 << 10)
CAM_LRME (1 << 11)
CAM_FLASH (1 << 12)
CAM_ACTUATOR (1 << 13)
CAM_CCI (1 << 14)
CAM_CSIPHY (1 << 15)
CAM_EEPROM (1 << 16)
CAM_UTIL (1 << 17)
CAM_HFI (1 << 18)
CAM_CTXT (1 << 19)
CAM_OIS (1 << 20)
CAM_RES (1 << 21)
CAM_MEM (1 << 22) /*Memory manager logs*/ CAM_IRQ_CTRL (1 << 23)
CAM_REQ (1 << 24) /*Tracks a request across kernel drivers*/ CAM_PERF (1 << 25)
/*Performance related logs i.e. clk, BW vote etc*/ CAM_CUSTOM (1 << 26)
CAM_PRESENSE (1 << 27)
CAM_OPE (1 << 28)
CAM_IO_ACCESS (1 << 29)
CAM_SFE (1 << 30)
CAM_IR_LED (1 << 31)
```

6 References

6.1 Related documents

Document
Qualcomm Linux Build Guide
Qualcomm Linux Camera Guide - Addendum
Qualcomm Linux Migration Guide
Qualcomm IM SDK Reference

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“Qualcomm Technologies”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “Qualcomm Internal Use Only”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

THE DOCUMENTATION ACCOMPANYING THE MATERIALS AND/OR RELEVANT PRODUCTS AND SERVICE OFFERINGS MAY INCLUDE IMPORTANT USE LIMITATIONS. ANY DEVIATIONS FROM APPLICABLE USE LIMITATIONS MAY ADVERSELY IMPACT PERFORMANCE, DURABILITY, QUALITY OR SAFETY. YOU ASSUME ALL RISKS AND LIABILITIES ASSOCIATED WITH ANY DEVIATIONS.